Key Points

- Research suggests adding `OH_Mar13_N2_Re_fit` and `Av_gas_Re` can improve SFR predictions by capturing metallicity and dust effects.
- It seems likely that interaction terms (e.g., `log_Mass_gas * nsa_mstar`) and nonlinear terms (e.g., `log_Mass_gas^2`) will enhance model fit, given `log_Mass_gas`'s dominance.
- The evidence leans toward starting with current data, but consider mangaHIall or Gema2 for gas details if needed.

Direct Answer

To implement and test the Random Forest model using "pipe3d_data2.csv" with additional data points and transformations, follow these steps for a clear, effective approach:

Data Preparation

First, load your dataset "pipe3d_data2.csv" into Python. This file likely contains galaxy properties like gas mass, stellar mass, and SFR, which are key for predicting star formation rates (SFR). We'll focus on including `OH_Mar13_N2_Re_fit` (metallicity) and `Av_gas_Re` (dust attenuation) to improve predictions, as these capture how metal content and dust affect star formation.

Feature Engineering

Since `log_Mass_gas` is a major predictor, create new features to capture complex relationships:

- Add interaction terms like `log_Mass_gas * nsa_mstar` to see how gas and stellar mass work together.
- Include nonlinear terms like `log_Mass_gas^2` to model curved effects.
- Also, try combinations like `log_Mass_gas * OH_Mar13_N2_Re_fit` to explore how metallicity influences the gas-SFR link.

Model Training and Testing

Split the data into training (80%) and testing (20%) sets to ensure fair evaluation. Train the Random Forest model on the training set with these new features, using cross-validation to check performance. Evaluate using Mean Squared Error (MSE) and R-squared (R^2) to see if the model improves over your previous results (e.g., MSE around 0.27, R^2 around 0.82). Check which features are most important and look at residual plots to see if the fit is better, especially for outliers.

Unexpected Detail: Quality Control

Your dataset includes a `QCFLAG` column, which can filter for valid data (e.g., `QCFLAG == 1`), potentially boosting model accuracy—an often overlooked step in astronomical data analysis.
If the models don't improve enough, you have mangaHIall or Gema2 catalogue CSV files for more gas details. You can consider merging these later, but start with the current data to see how far you can get.
Here's a script to get you started:

Test 4: Random Forest Model

```
python

import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt

# Load the dataset
df = pd.read_csv('pipe3d_data2.csv')
print(f"Dataset loaded with {len(df)} rows and {len(df.columns)} columns.")

# Define target and base features
target = 'log_SFR_Ha'
base_features = ['log_Mass_gas', 'nsa_mstar', 'log_Mass', 'V-band_SB_at_Re', 'vel_sigma_Re',
'OH_Mar13_N2_Re_fit', 'Av_gas_Re']

# Impute missing values with median
imputer = SimpleImputer(strategy='median')
df[base_features] = imputer.fit_transform(df[base_features])

# Drop rows with missing target
df = df.dropna(subset=[target])
y = df[target]
X = df[base_features]

# Feature engineering: Create interactions and polynomials
df['log_Mass_gas_times_nsa_mstar'] = df['log_Mass_gas'] * df['nsa_mstar']
df['log_Mass_gas_times_metallicity'] = df['log_Mass_gas'] * df['OH_Mar13_N2_Re_fit']
df['log_Mass_gas_squared'] = df['log_Mass_gas'] ** 2
enhanced_features = base_features + ['log_Mass_gas_times_nsa_mstar',
'log_Mass_gas_times_metallicity', 'log_Mass_gas_squared']

# Use PolynomialFeatures for systematic generation (degree 2)
```

```
poly = PolynomialFeatures(degree=2, include_bias=False, interaction_only=False)
X_poly = poly.fit_transform(df[enhanced_features])
feature_names = poly.get_feature_names_out(enhanced_features)

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X_poly, y, test_size=0.2, random_state=42)
print(f"Training set: {X_train.shape[0]} rows, Testing set: {X_test.shape[0]} rows")

# Train Random Forest with cross-validation
rf = RandomForestRegressor(random_state=42)
cv_scores = cross_val_score(rf, X_train, y_train, cv=5, scoring='neg_mean_squared_error')
print(f"Cross-validation MSE scores: {(-cv_scores).mean():.4f} (+/- {(-cv_scores).std() * 2:.4f})")

# Fit the model
rf.fit(X_train, y_train)

# Predict and evaluate
y_pred = rf.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f"Test MSE: {mse:.4f}")
print(f"Test R-squared: {r2:.4f}")

# Feature importance
importances = rf.feature_importances_
importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': importances})
importance_df = importance_df.sort_values(by='Importance', ascending=False)
print("\nTop 10 Important Features:")
print(importance_df.head(10))

# Plot predicted vs actual
plt.figure(figsize=(8,6))
plt.scatter(y_test, y_pred, alpha=0.5)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
plt.xlabel('Actual log_SFR_Ha')
plt.ylabel('Predicted log_SFR_Ha')
plt.title('Predicted vs Actual log_SFR_Ha')
plt.savefig('predicted_vs_actual.png')
plt.close()

# Plot residuals
residuals = y_test - y_pred
plt.figure(figsize=(8,6))
plt.scatter(y_pred, residuals, alpha=0.5)
plt.axhline(0, color='r', linestyle='--')
plt.xlabel('Predicted log_SFR_Ha')
plt.ylabel('Residuals')
plt.title('Residuals vs Predicted')
plt.savefig('residuals.png')
```

```
plt.close()

# Plot histogram of residuals
plt.figure(figsize=(8,6))
plt.hist(residuals, bins=30, edgecolor='black')
plt.xlabel('Residuals')
plt.ylabel('Frequency')
plt.title('Histogram of Residuals')
plt.savefig('residuals_histogram.png')
plt.close()
```

This script starts with your current data, includes new features, and tests them thoroughly. If you need more gas details later, you can merge mangaHIall or Gema2, but this should get you far.

Comprehensive Analysis and Process

This section provides a detailed exploration of implementing and testing the Random Forest model with "pipe3d_data2.csv", focusing on including additional data points like `OH_Mar13_N2_Re_fit` and `Av_gas_Re`, and exploring interaction and nonlinear terms, particularly involving `log_Mass_gas`, given its dominance as a predictor. It integrates insights from prior analyses, feature importance charts, and the availability of mangaHIall or Gema2 catalogues, ensuring a thorough approach.

Background and Context

Your project involves predicting star formation rates (SFR) using Random Forest and Gradient Boosting models, with scatter plots (Images 0, 5, 7) showing good alignment along the 1:1 line but with outliers and residual skewness (Images 1, 3, 4, 6, 9). The feature importance charts (Images 2, 8) highlight `log_Mass_gas` as the dominant predictor (~0.7-0.8 importance), with secondary contributions from `nsa_mstar` and `log_Mass`. The current dataset, derived from the SDSS17Pipe3Dv1_1 catalogue, includes 536 columns and 15,000 rows, now filtered to 10,081 rows in your current analysis, with key columns like `log_SFR_Ha`, `log_Mass_gas`, and various flux measurements. You also have access to mangaHIall or Gema2 catalogue CSV files for gas details if needed, but you want to start with the current data, specifically "pipe3d_data2.csv".

The goal is to implement and test the Random Forest model by including additional data points and exploring interaction and nonlinear terms, especially involving `log_Mass_gas`, to improve model fit and address residual skewness and outliers.

Detailed Implementation and Testing Strategy

1. Data Preparation

- **Load the Dataset**: Start with "pipe3d_data2.csv", assuming it's a filtered version of the SDSS17Pipe3Dv1_1 catalogue, likely including `log_SFR_Ha`, `log_Mass_gas`, `nsa_mstar`, etc. Verify column presence:

    - `df = pd.read_csv('pipe3d_data2.csv')`
    - Check for `OH_Mar13_N2_Re_fit` and `Av_gas_Re` using `assert 'OH_Mar13_N2_Re_fit' in df.columns and 'Av_gas_Re' in df.columns`.
- **Handle Missing Values**: Replace sentinel values (-9999) with NaN and impute missing values or drop rows. Use median imputation for robustness:
    - `df[['OH_Mar13_N2_Re_fit', 'Av_gas_Re']] = df[['OH_Mar13_N2_Re_fit', 'Av_gas_Re']].replace(-9999, np.nan)`
    - `from sklearn.impute import SimpleImputer; imputer = SimpleImputer(strategy='median'); df[['OH_Mar13_N2_Re_fit', 'Av_gas_Re']] = imputer.fit_transform(df[['OH_Mar13_N2_Re_fit', 'Av_gas_Re']])`
- **Quality Control**: Use `QCFLAG` if available to filter for valid data (e.g., `df = df[df['QCFLAG'] == 1]`), enhancing model robustness, an often overlooked step in astronomical data analysis Pandas Documentation.

2. Feature Engineering

Given `log_Mass_gas`'s dominance (~0.7-0.8 importance, Images 2, 8), explore interaction and nonlinear terms to capture complex relationships, addressing residual skewness (Images 1, 3, 4, 6, 9).

- **Base Features**: Start with key predictors: `log_Mass_gas`, `nsa_mstar`, `log_Mass`, `V-band_SB_at_Re`, `vel_sigma_Re`, and now include `OH_Mar13_N2_Re_fit`, `Av_gas_Re`.
- **Interaction Terms**:
    - Create `log_Mass_gas * nsa_mstar` to capture combined gas-stellar mass effects: `df['log_Mass_gas_times_nsa_mstar'] = df['log_Mass_gas'] * df['nsa_mstar']`.
    - Include `log_Mass_gas * OH_Mar13_N2_Re_fit` to model metallicity's influence: `df['log_Mass_gas_times_metallicity'] = df['log_Mass_gas'] * df['OH_Mar13_N2_Re_fit']`.
    - Add `log_Mass_gas * Av_gas_Re` for dust effects: `df['log_Mass_gas_times_Av'] = df['log_Mass_gas'] * df['Av_gas_Re']`.
- **Nonlinear Terms**:
    - Generate `log_Mass_gas^2` to capture quadratic effects: `df['log_Mass_gas_squared'] = df['log_Mass_gas'] ** 2`.
    - Consider higher-order terms if needed, but start with degree 2 for simplicity.
- **Systematic Generation**: Use scikit-learn's `PolynomialFeatures` for automated creation:
    - `from sklearn.preprocessing import PolynomialFeatures`
    - `poly = PolynomialFeatures(degree=2, include_bias=False, interaction_only=False)`
    - `features = ['log_Mass_gas', 'nsa_mstar', 'OH_Mar13_N2_Re_fit', 'Av_gas_Re', 'log_Mass', 'V-band_SB_at_Re', 'vel_sigma_Re']`

    - `X = df[features]; X_poly = poly.fit_transform(X); feature_names = poly.get_feature_names_out(features)`
    - `df_poly = pd.DataFrame(X_poly, columns=feature_names)`
- **Consider Other Features**: From the list, include `Age_LW_Re_fit` (stellar age) or `vel_disp_ssp_1Re` (velocity dispersion) if they improve fit, checking for correlations to avoid redundancy:
    - Example: `df.corr()[['log_Mass_gas', 'OH_Mar13_N2_Re_fit', 'Av_gas_Re']].abs().sort_values(by='log_Mass_gas', ascending=False).`

3. Model Implementation and Testing

- **Train Random Forest**: Use the enhanced feature set with cross-validation for robustness.
- **Example Code** (as provided in the script):
    - Initialize: `rf = RandomForestRegressor(random_state=42)`
    - Fit with cross-validation: `cv_scores = cross_val_score(rf, X_train, y_train, cv=5, scoring='neg_mean_squared_error')`
    - Train on full training set: `rf.fit(X_train, y_train)`
    - Predict and evaluate: `y_pred = rf.predict(X_test); mse = mean_squared_error(y_test, y_pred); r2 = r2_score(y_test, y_pred)`
- **Hyperparameter Tuning**: Optimize using GridSearchCV, focusing on `n_estimators`, `max_depth`, `min_samples_split`:
    - `param_grid = {'n_estimators': [100, 200, 300], 'max_depth': [10, 20, None], 'min_samples_split': [2, 5, 10]}`
    - `grid_search = GridSearchCV(rf, param_grid, cv=5, scoring='neg_mean_squared_error', n_jobs=-1); grid_search.fit(X_train, y_train)`
- **Evaluation**:
    - Compare with baseline (e.g., MSE ~0.27, R^2 ~0.82 from prior results).
    - Analyze feature importance: `importances = rf.feature_importances_; importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': importances}).sort_values(by='Importance', ascending=False)`
    - Plot residuals and histograms as shown in the script for diagnostic insights.

4. Verification and Robustness

- **Physical Alignment**: Compare predictions to the Kennicutt-Schmidt law Kennicutt-Schmidt Law 1998, ensuring `log_Mass_gas` and new features align with astrophysical expectations.
- **External Testing**: Validate on MaNGA data MaNGA Data Release 17 for generalizability, especially for metallicity and dust effects.

- **Statistical Robustness**: Test residual normality using Shapiro-Wilk test: `from scipy.stats import shapiro; stat, p = shapiro(residuals); print('Shapiro-Wilk test p-value:', p)`. Use permutation importance to verify feature significance, especially for `log_Mass_gas` and new terms.

5. Consideration of Additional Datasets

- **mangaHIall**: Likely contains HI (neutral hydrogen) data, directly related to gas content. Since `log_Mass_gas` is already a feature, and presumably derived from HI, adding mangaHIall might not provide new information unless it offers higher resolution or molecular gas data, enhancing `log_Mass_gas`. Research suggests HI data can improve gas mass estimates HI in Galaxies 2012, but start with current data first.
- **Gema2**: Without specifics, it's unclear, but if it provides unique features (e.g., molecular gas, different metallicity calibrations), it could be beneficial. Given the richness of SDSS17Pipe3Dv1_1, it's likely sufficient initially.
- **Recommendation**: Focus on current data first. If model performance plateaus (e.g., MSE > 0.25, $R^2 < 0.83$), consider merging mangaHIall or Gema2 based on common identifiers (e.g., `mangaid`), ensuring consistency.

Implementation Details

The provided script implements this strategy, starting with "pipe3d_data2.csv", including new features, and testing them thoroughly. It uses cross-validation, feature engineering, and visualization to ensure robust evaluation, addressing your need to improve model fit, especially for outliers and residual skewness.

Conclusion

By implementing and testing with the current dataset, including `OH_Mar13_N2_Re_fit`, `Av_gas_Re`, and exploring interaction and nonlinear terms involving `log_Mass_gas`, you can refine your SFR prediction models. Start with the provided script, and only consider mangaHIall or Gema2 if needed, ensuring a data-driven approach to model improvement.

Key Citations

- Scikit-learn Documentation
- Pandas Documentation
- Matplotlib Documentation
- Kennicutt-Schmidt Law 1998
- MaNGA Data Release 17
- HI in Galaxies 2012

Results

```
(myenv) root@BigsbyGaming:/home/pmqr7# python test4_RF.py
Dataset loaded with 10220 rows and 536 columns.
Training set: 8084 rows, Testing set: 2022 rows
Cross-validation MSE scores: 0.2802 (+/- 0.0466)
Test MSE: 0.2742
Test R-squared: 0.8257

Top 10 Important Features:
                                  Feature  Importance
19               log_Mass_gas log_Mass_gas_squared    0.155789
10                           log_Mass_gas^2    0.127827
0                             log_Mass_gas    0.115442
64                log_Mass_gas_squared^2    0.108569
9                  log_Mass_gas_squared    0.084118
53  OH_Mar13_N2_Re_fit log_Mass_gas_times_metallicity    0.058430
63  log_Mass_gas_times_metallicity log_Mass_gas_sq...    0.028310
36                log_Mass log_Mass_gas_squared    0.026051
21                     nsa_mstar log_Mass    0.019797
34           log_Mass log_Mass_gas_times_nsa_mstar    0.017628
```

Key Points

- Research suggests adding `OH_Mar13_N2_Re_fit` and `Av_gas_Re` can improve SFR predictions by capturing metallicity and dust effects.
- It seems likely that interaction terms (e.g., `log_Mass_gas * nsa_mstar`) and nonlinear terms (e.g., `log_Mass_gas^2`) will enhance model fit, given `log_Mass_gas`'s dominance.
- The evidence leans toward starting with current data, but consider mangaHIall or Gema2 for gas details if needed.

Direct Answer

Dataset and Model Overview

Your Random Forest model, tested on "pipe3d_data2.csv" with 10,220 rows and 536 columns, achieved a test Mean Squared Error (MSE) of 0.2742 and an R-squared (R^2) of 0.8257. This means it explains about 82.57% of the variance in star formation rates (SFR), which is a strong result. The model used cross-validation, showing an average MSE of 0.2802 (±0.0466), indicating consistent performance.

Feature Importance and Enhancements

The top features are mostly related to `log_Mass_gas`, like `log_Mass_gas^3` (importance 0.1558) and `log_Mass_gas^2` (0.1278), showing a nonlinear relationship. New features like `OH_Mar13_N2_Re_fit log_Mass_gas_times_metallicity` (0.0584) also contribute, suggesting metallicity helps. To refine, add interaction terms (e.g., `log_Mass_gas * nsa_mstar`) and nonlinear terms (e.g., `log_Mass_gas^2`), already included here, to capture complex effects.

Testing and Next Steps

Split the data into training (80%, 8,084 rows) and testing (20%, 2,022 rows) sets for fair evaluation. Use cross-validation to ensure robustness, and compare metrics to see if new features improve fit, especially for outliers. Check residual plots to assess improvements, and consider Gradient Boosting if you want potentially better accuracy, as it previously showed lower MSE.

Unexpected Detail: Quality Control

Your dataset includes a `QCFLAG` column, which can filter for valid data (e.g., `QCFLAG == 1`), potentially boosting accuracy—an often overlooked step in astronomical data analysis.
If performance plateaus, you have mangaHIall or Gema2 for more gas details, but start with current data to see how far you can get.

Survey Note: Comprehensive Analysis and Implementation

This section provides a detailed exploration of implementing and testing the Random Forest model using "pipe3d_data2.csv," focusing on including additional data points and exploring interaction and nonlinear terms, particularly involving `log_Mass_gas`, given its dominance as a predictor. It integrates insights from the provided results, feature importance charts, and the availability of mangaHIall or Gema2 catalogues, ensuring a thorough approach.

Background and Context

Your project involves predicting star formation rates (SFR) using machine learning models, with a focus on Random Forest and Gradient Boosting, as evidenced by prior scatter plots (Images 0, 5, 7) and residual analyses (Images 1, 3, 4, 6, 9). The feature importance charts (Images 2, 8) highlight `log_Mass_gas` as the dominant predictor (~0.7-0.8 importance), with secondary contributions from `nsa_mstar` and `log_Mass`. The current dataset, "pipe3d_data2.csv," derived from the SDSS17Pipe3Dv1_1 catalogue, includes 536 columns and 10,220 rows, with key columns like `log_SFR_Ha`, `log_Mass_gas`, and various flux

measurements. You also have access to mangaHIall or Gema2 catalogue CSV files for gas details if needed, but you want to start with the current data.
The goal is to implement and test the Random Forest model by including additional data points and exploring interaction and nonlinear terms, especially involving `log_Mass_gas`, to improve model fit and address residual skewness and outliers, based on the provided test results.

Dataset and Model Results Analysis

The output from `test4_RF.py` provides the following metrics:

- **Dataset Size**: 10,220 rows, 536 columns, split into 8,084 training rows (80%) and 2,022 testing rows (20%).
- **Cross-Validation Performance**: 5-fold cross-validation MSE of 0.2802 (±0.0466), indicating consistent performance with a standard deviation suggesting stable generalization.
- **Test Performance**: Test MSE of 0.2742, R-squared of 0.8257, showing the model explains 82.57% of the variance in SFR, a strong result for regression tasks.
- **Feature Importance**: The top 10 features, sorted by importance, are:

Rank	Feature	Importance
1	log_Mass_gas log_Mass_gas_squared	0.155789
2	log_Mass_gas^2	0.127827
3	log_Mass_gas	0.115442
4	log_Mass_gas_squared^2	0.108569
5	log_Mass_gas_squared	0.084118
6	OH_Mar13_N2_Re_fit log_Mass_gas_times_metallicity	0.058430
7	log_Mass_gas_times_metallicity log_Mass_gas_squared	0.028310
8	log_Mass log_Mass_gas_squared	0.026051
9	nsa_mstar log_Mass	0.019797
10	log_Mass log_Mass_gas_times_nsa_mstar	0.017628

Interpretation:

- The dominance of `log_Mass_gas`-related terms (e.g., `log_Mass_gas^3`, `log_Mass_gas^2`, `log_Mass_gas^4`) suggests a strong nonlinear relationship, aligning with astrophysical expectations like the Kennicutt-Schmidt law Kennicutt-Schmidt Law 1998.

- The inclusion of `OH_Mar13_N2_Re_fit log_Mass_gas_times_metallicity` (0.0584) indicates metallicity's role, supporting the addition of such features.
- The slight improvement in R^2 (0.8257 vs. previous 0.8217) with polynomial features suggests marginal benefit, but cross-validation MSE (0.2802) is close to test MSE (0.2742), indicating no significant overfitting.

Implementation and Testing Strategy

To implement and test the Random Forest model with "pipe3d_data2.csv," including additional data points and transformations, follow these steps:

1. Data Preparation

- **Load the Dataset**: Ensure "pipe3d_data2.csv" is in your working directory and load it:
  - `df = pd.read_csv('pipe3d_data2.csv')`
  - Verify column presence for key features: `log_Mass_gas`, `nsa_mstar`, `OH_Mar13_N2_Re_fit`, `Av_gas_Re`, etc.
- **Handle Missing Values**: Replace sentinel values (-9999) with NaN and impute or drop rows:
  - `df = df.replace(-9999, np.nan)`
  - Use `SimpleImputer` for median imputation: `imputer = SimpleImputer(strategy='median'); df[['OH_Mar13_N2_Re_fit', 'Av_gas_Re']] = imputer.fit_transform(df[['OH_Mar13_N2_Re_fit', 'Av_gas_Re']])`
- **Quality Control**: Filter using `QCFLAG` if available: `df = df[df['QCFLAG'] == 1]`, enhancing robustness Pandas Documentation.

2. Feature Engineering

Given `log_Mass_gas`'s dominance, enhance features to capture complex relationships:

- **Base Features**: Include `log_Mass_gas`, `nsa_mstar`, `log_Mass`, `V-band_SB_at_Re`, `vel_sigma_Re`, `OH_Mar13_N2_Re_fit`, `Av_gas_Re`.
- **Interaction Terms**: Create manually:
  - `df['log_Mass_gas_times_nsa_mstar'] = df['log_Mass_gas'] * df['nsa_mstar']`
  - `df['log_Mass_gas_times_metallicity'] = df['log_Mass_gas'] * df['OH_Mar13_N2_Re_fit']`
  - `df['log_Mass_gas_times_Av'] = df['log_Mass_gas'] * df['Av_gas_Re']`
- **Nonlinear Terms**: Add:
  - `df['log_Mass_gas_squared'] = df['log_Mass_gas'] ** 2`
- **Systematic Generation**: Use `PolynomialFeatures` for degree 2:
  - `poly = PolynomialFeatures(degree=2, include_bias=False, interaction_only=False)`

    - `features = ['log_Mass_gas', 'nsa_mstar', 'OH_Mar13_N2_Re_fit', 'Av_gas_Re', 'log_Mass', 'V-band_SB_at_Re', 'vel_sigma_Re', 'log_Mass_gas_times_nsa_mstar', 'log_Mass_gas_times_metallicity', 'log_Mass_gas_squared']`
    - `X_poly = poly.fit_transform(df[features]); feature_names = poly.get_feature_names_out(features)`

3. Model Training and Testing

- **Split Data**: Use 80-20 split for training and testing:
    - `X_train, X_test, y_train, y_test = train_test_split(X_poly, df['log_SFR_Ha'], test_size=0.2, random_state=42)`
- **Train Random Forest**: Use cross-validation for robustness:
    - `rf = RandomForestRegressor(random_state=42)`
    - `cv_scores = cross_val_score(rf, X_train, y_train, cv=5, scoring='neg_mean_squared_error'); print(f"Cross-validation MSE scores: {(-cv_scores).mean():.4f} (+/- {(-cv_scores).std() * 2:.4f})")`
    - Fit: `rf.fit(X_train, y_train)`
- **Evaluate**: Predict and compute metrics:
    - `y_pred = rf.predict(X_test); mse = mean_squared_error(y_test, y_pred); r2 = r2_score(y_test, y_pred)`
    - Print: `print(f"Test MSE: {mse:.4f}\nTest R-squared: {r2:.4f}")`
- **Feature Importance**: Extract and display top 10:
    - `importances = rf.feature_importances_; importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': importances}).sort_values(by='Importance', ascending=False); print("\nTop 10 Important Features:\n", importance_df.head(10))`
- **Visualizations**: Plot predicted vs. actual and residuals:
    - `plt.scatter(y_test, y_pred); plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--'); plt.savefig('predicted_vs_actual.png'); plt.close()`
    - `residuals = y_test - y_pred; plt.scatter(y_pred, residuals); plt.axhline(0, color='r', linestyle='--'); plt.savefig('residuals.png'); plt.close()`
    - `plt.hist(residuals, bins=30, edgecolor='black'); plt.savefig('residuals_histogram.png'); plt.close()`

4. Verification and Robustness

- **Physical Alignment**: Compare predictions to Kennicutt-Schmidt law [Kennicutt-Schmidt Law 1998](https://ui.adsabs.harvard.edu/abs/1998ApJ...498..541

Test 4: Gradient Boosting mode

Key Points

- Research suggests adding `OH_Mar13_N2_Re_fit` and `Av_gas_Re` can improve SFR predictions by capturing metallicity and dust effects.
- It seems likely that interaction terms (e.g., `log_Mass_gas * nsa_mstar`) and nonlinear terms (e.g., `log_Mass_gas^2`) will enhance model fit, given `log_Mass_gas`'s dominance.
- The evidence leans toward starting with current data, but consider mangaHIall or Gema2 for gas details if needed.

Direct Answer

Below is a complete, clear, and concise Python script using the Gradient Boosting model with "pipe3d_data2.csv" to predict star formation rates (SFR). It includes new features like metallicity (`OH_Mar13_N2_Re_fit`) and dust attenuation (`Av_gas_Re`), and explores interaction and nonlinear terms, especially for `log_Mass_gas`, which is a key predictor.

Script Overview

This script loads your dataset, prepares features, trains a Gradient Boosting model, evaluates its performance, and visualizes results. It handles missing values, creates polynomial features, and uses cross-validation for robustness.

Data Preparation

- Load "pipe3d_data2.csv" and focus on key features like gas mass, stellar mass, and new additions for better predictions.
- Fill in any missing values to keep the data clean, and split it into training (80%) and testing (20%) sets.

Feature Engineering

- Add interaction terms (e.g., `log_Mass_gas * nsa_mstar`) and nonlinear terms (e.g., `log_Mass_gas^2`) to capture complex relationships, given `log_Mass_gas`'s importance.
- Use a tool to automatically generate these features, ensuring the model can learn from them.

Model Training and Testing

- Train the Gradient Boosting model on the training set, using cross-validation to check how well it performs.
- Evaluate on the test set with metrics like Mean Squared Error (MSE) and R-squared (R^2), comparing to previous results (e.g., MSE around 0.26, R^2 around 0.83).
- Look at which features matter most and check plots to see if the fit improves, especially for outliers.

Unexpected Detail: Quality Control

Your dataset includes a `QCFLAG` column, which can filter for valid data (e.g., `QCFLAG == 1`), potentially boosting model accuracy—an often overlooked step in astronomical data analysis.

Here's the script:

```
Test

python
import pandas as pd
import numpy as np
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt

# Load the dataset
df = pd.read_csv('pipe3d_data2.csv')
print(f"Dataset loaded with {len(df)} rows and {len(df.columns)} columns.")

# Define target and base features
target = 'log_SFR_Ha'
base_features = ['log_Mass_gas', 'nsa_mstar', 'log_Mass', 'V-band_SB_at_Re', 'vel_sigma_Re',
'OH_Mar13_N2_Re_fit', 'Av_gas_Re']

# Impute missing values with median
imputer = SimpleImputer(strategy='median')
df[base_features] = imputer.fit_transform(df[base_features])

# Drop rows with missing target
df = df.dropna(subset=[target])
y = df[target]
X = df[base_features]

# Apply PolynomialFeatures for degree 2 to include interactions and squares
poly = PolynomialFeatures(degree=2, include_bias=False)
```

```
X_poly = poly.fit_transform(X)
feature_names = poly.get_feature_names_out(base_features)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_poly, y, test_size=0.2, random_state=42)
print(f"Training set: {X_train.shape[0]} rows, Testing set: {X_test.shape[0]} rows")

# Initialize and train Gradient Boosting Regressor
gb = GradientBoostingRegressor(n_estimators=100, learning_rate=0.1, max_depth=3, random_state=42)

# Perform cross-validation
cv_scores = cross_val_score(gb, X_train, y_train, cv=5, scoring='neg_mean_squared_error')
print(f"Cross-validation MSE: {(-cv_scores).mean():.4f} (+/- {(-cv_scores).std() * 2:.4f})")

# Fit the model on the entire training set
gb.fit(X_train, y_train)

# Predict on the test set
y_pred = gb.predict(X_test)

# Calculate evaluation metrics
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f"Test MSE: {mse:.4f}")
print(f"Test R-squared: {r2:.4f}")

# Feature importances
importances = gb.feature_importances_
importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': importances})
importance_df = importance_df.sort_values(by='Importance', ascending=False)
print("\nTop 10 Important Features:")
print(importance_df.head(10))

# Plot predicted vs actual
plt.figure(figsize=(8,6))
plt.scatter(y_test, y_pred, alpha=0.5)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
plt.xlabel('Actual log_SFR_Ha')
plt.ylabel('Predicted log_SFR_Ha')
plt.title('Predicted vs Actual log_SFR_Ha')
plt.savefig('predicted_vs_actual_gb.png')
plt.close()

# Plot residuals vs predicted
residuals = y_test - y_pred
plt.figure(figsize=(8,6))
plt.scatter(y_pred, residuals, alpha=0.5)
plt.axhline(0, color='r', linestyle='--')
plt.xlabel('Predicted log_SFR_Ha')
```

```
plt.ylabel('Residuals')
plt.title('Residuals vs Predicted')
plt.savefig('residuals_gb.png')
plt.close()

# Plot histogram of residuals
plt.figure(figsize=(8,6))
plt.hist(residuals, bins=30, edgecolor='black')
plt.xlabel('Residuals')
plt.ylabel('Frequency')
plt.title('Histogram of Residuals')
plt.savefig('residuals_histogram_gb.png')
plt.close()
```

This script starts with your current data, includes new features, and tests them thoroughly. If you need more gas details later, you can merge mangaHIall or Gema2, but this should get you far. For better results, consider tuning hyperparameters or adding more features if performance plateaus.

Results

```
(myenv) root@BigsbyGaming:/home/pmqr7# python test4_GP.py
Dataset loaded with 10220 rows and 536 columns.
Training set: 8084 rows, Testing set: 2022 rows
Cross-validation MSE: 0.2732 (+/- 0.0491)
Test MSE: 0.2457
Test R-squared: 0.8438

Top 10 Important Features:
                       Feature  Importance
12  log_Mass_gas OH_Mar13_N2_Re_fit    0.427977
0                   log_Mass_gas    0.271795
7                  log_Mass_gas^2    0.119301
15              nsa_mstar log_Mass    0.042193
9            log_Mass_gas log_Mass    0.021879
13           log_Mass_gas Av_gas_Re    0.018806
18     nsa_mstar OH_Mar13_N2_Re_fit    0.015090
34                   Av_gas_Re^2    0.011188
33     OH_Mar13_N2_Re_fit Av_gas_Re    0.010054
2                     log_Mass    0.007132
```

Key Points

- Research suggests adding `OH_Mar13_N2_Re_fit` and `Av_gas_Re` can improve SFR predictions by capturing metallicity and dust effects.
- It seems likely that interaction terms (e.g., `log_Mass_gas * nsa_mstar`) and nonlinear terms (e.g., `log_Mass_gas^2`) will enhance model fit, given `log_Mass_gas`'s dominance.
- The evidence leans toward starting with current data, but consider mangaHIall or Gema2 for gas details if needed.

Interpreting Results and Next Steps

Model Performance

The Gradient Boosting model, tested on a dataset of 10,220 rows and 536 columns, shows strong performance with a test Mean Squared Error (MSE) of 0.2457 and an R-squared (R^2) of 0.8438. This means it explains about 84.38% of the variance in star formation rates (SFR), which is excellent for predicting how stars form in galaxies. The cross-validation MSE of 0.2732 (±0.0491) suggests the model is consistent across different data splits, with a small standard deviation indicating stable generalization.

Feature Importance

The top features highlight the importance of `log_Mass_gas` and its interactions. For example, `log_Mass_gas * OH_Mar13_N2_Re_fit` is the most important (0.427977), showing that combining gas mass with metallicity is key. Other significant features include `log_Mass_gas^2` (0.119301) and `nsa_mstar * log_Mass` (0.042193), suggesting nonlinear and interaction effects are crucial. This aligns with astrophysical ideas, like how gas and metal content together influence star formation.

Comparing with Random Forest

Compared to the Random Forest model (test MSE 0.2742, R^2 0.8257), Gradient Boosting performs better, with a lower MSE and higher R^2. This means Gradient Boosting captures the data's patterns more effectively, making it a stronger choice for now.

Unexpected Detail: Quality Control

Your dataset includes a `QCFLAG` column, which can filter for valid data (e.g., `QCFLAG == 1`), potentially boosting accuracy—an often overlooked step in astronomical data analysis.

Next Steps

To refine further, consider:

- **Tuning Hyperparameters**: Adjust settings like learning rate or tree depth to potentially lower MSE.
- **Checking Residuals**: Look at prediction errors to see if the model struggles with specific galaxies, guiding improvements.
- **Validating Externally**: Test on another dataset to ensure it works well beyond your current data.
- **Adding More Data**: If needed, use mangaHIall or Gema2 for detailed gas info, but start with what you have.

This approach keeps things simple and builds on your strong results, ensuring reliable SFR predictions.

Comprehensive Analysis and Interpretation

This section provides a detailed exploration of the Gradient Boosting model results from `test4_GP.py`, integrating insights from prior analyses, feature importance charts, and the availability of mangaHIall or Gema2 catalogues, ensuring a thorough approach to interpreting performance and planning next steps.

Background and Context

Your project involves predicting star formation rates (SFR) using machine learning models, with a focus on Random Forest and Gradient Boosting, as evidenced by prior scatter plots (Images 0, 5, 7) and residual analyses (Images 1, 3, 4, 6, 9). The feature importance charts (Images 2, 8) highlight `log_Mass_gas` as the dominant predictor (~0.7-0.8 importance), with secondary contributions from `nsa_mstar` and `log_Mass`. The current dataset, "pipe3d_data2.csv," derived from the SDSS17Pipe3Dv1_1 catalogue, includes 536 columns and 10,220 rows, with key columns like `log_SFR_Ha`, `log_Mass_gas`, and various flux measurements. You also have access to mangaHIall or Gema2 catalogue CSV files for gas details if needed, but you want to start with the current data.

The goal is to interpret the Gradient Boosting model results, compare with Random Forest, and suggest next steps for refinement, including potential use of additional datasets.

Dataset and Model Results Analysis

The output from `test4_GP.py` provides the following metrics:

- **Dataset Size**: 10,220 rows, 536 columns, split into 8,084 training rows (80%) and 2,022 testing rows (20%).
- **Cross-Validation Performance**: 5-fold cross-validation MSE of 0.2732 (±0.0491), indicating consistent performance with a standard deviation of 0.0491, suggesting stable generalization across folds.
- **Test Performance**: Test MSE of 0.2457, R-squared of 0.8438, showing the model explains 84.38% of the variance in SFR, a strong result for regression tasks.

- **Feature Importance**: The top 10 features, sorted by importance, are:

Rank	Feature	Importance
1	log_Mass_gas OH_Mar13_N2_Re_fit	0.427977
2	log_Mass_gas	0.271795
3	log_Mass_gas^2	0.119301
4	nsa_mstar log_Mass	0.042193
5	log_Mass_gas log_Mass	0.021879
6	log_Mass_gas Av_gas_Re	0.018806
7	nsa_mstar OH_Mar13_N2_Re_fit	0.015090
8	Av_gas_Re^2	0.011188
9	OH_Mar13_N2_Re_fit Av_gas_Re	0.010054
10	log_Mass	0.007132

Interpretation:

- The dominance of `log_Mass_gas * OH_Mar13_N2_Re_fit` (0.427977) suggests that the interaction between gas mass and metallicity is the most critical predictor, aligning with astrophysical theory where metallicity affects gas cooling and star formation efficiency [Marino et al. (2013)](https://ui.adsabs.harvard.edu/abs/201

Below is an interpretation of the provided visuals based on the context of our discussion about refining and verifying a theory using machine learning models—specifically Random Forest and Gradient Boosting—to predict star formation rates (SFR) in galaxies. The visuals include scatter plots, histograms of residuals, feature importance charts, and distributions of key variables. I'll break down each type of visual and relate them to our conversation about model performance, feature importance, and potential improvements.

---

1. Scatter Plots: Predicted vs. Actual SFR

- **Relevant Images**: 0, 5, 7, 13, 14, 17 (e.g., "Predicted vs Actual Star Formation Rates" and "Predicted vs Actual log SFR_Ha")

- **Description**: These plots compare predicted SFR (or log SFR_Ha) values against actual values for both Random Forest and Gradient Boosting models. The axes range from -5 to 2, with a red dashed line representing the ideal 1:1 relationship. Blue data points cluster around this line, with higher density between -2 and 1, and some scatter at the extremes (e.g., <-3 or >1).
- **Interpretation**:
    - **Model Performance**: The tight clustering along the diagonal line indicates that both models predict SFR accurately for most galaxies, supporting our goal of verifying the theory linking galaxy properties to SFR. The strong positive correlation suggests the models capture the underlying relationships well.
    - **Challenges at Extremes**: Outliers and wider scatter at low (<-3) and high (>1) SFR values highlight a limitation we discussed: the models struggle with extreme SFRs, possibly due to fewer training examples in these regions (as seen in Image 10). This suggests refining the models by addressing data imbalance or adding features to better handle these cases.

---

2. Histograms of Residuals

- **Relevant Images**: 1, 4, 6, 12, 15 (e.g., "Histogram of Residuals" for Random Forest and Gradient Boosting)
- **Description**: These histograms show the distribution of residuals (actual - predicted SFR) ranging from -2 to 6 (or -2 to 2 in some cases), with frequencies peaking around 0 (up to 700 in some plots). Random Forest residuals (e.g., Image 1) are slightly right-skewed, while Gradient Boosting residuals (e.g., Image 6) are more symmetric but still show a tail up to 6.
- **Interpretation**:
    - **Accuracy and Bias**: The peak at zero for both models indicates that predictions are generally close to actual values, aligning with our aim to refine a reliable predictive theory. However, the slight right skew in Random Forest residuals suggests a tendency to underpredict (positive residuals), while Gradient Boosting's more balanced distribution indicates better handling of bias—a point we noted when comparing model performance.
    - **Outliers**: Residuals extending to 6 (e.g., Images 1, 6) point to outliers where predictions deviate significantly, reinforcing our discussion about investigating extreme cases to improve robustness.

---

3. Feature Importance Charts

- **Relevant Images**: 2 ("Feature Importances from Random Forest"), 8 ("Feature Importances from Gradient Boosting")
- **Description**: These bar charts rank feature importance. For Random Forest (Image 2), `log_Mass_gas` dominates (0.7), followed by `nsa_mstar` (0.15) and `log_Mass` (0.12). For Gradient Boosting (Image 8), `log_Mass_gas` is again top (>0.8), with `nsa_mstar` (0.15) and interaction/nonlinear terms like `times_log_Mass` (~0.12) also significant.
- **Interpretation**:
    - **Key Drivers**: The dominance of `log_Mass_gas` in both models supports our hypothesis that gas mass is a critical driver of SFR, a cornerstone of the theory we're refining. The importance of `nsa_mstar` (stellar mass) further ties SFR to galaxy mass properties.
    - **Model Differences**: Gradient Boosting's emphasis on interaction terms (e.g., `times_log_Mass`) suggests nonlinear relationships we discussed exploring, potentially giving it an edge over Random Forest (which focuses on individual features). This aligns with our suggestion to incorporate complex feature interactions for better accuracy.

---

4. Residuals vs. Predicted Values Plots

- **Relevant Images**: 3 ("Residuals vs. Predicted Values (Random Forest)"), 9 ("Residuals vs. Predicted Values (Gradient Boosting)"), 16 ("Residuals vs Predicted")
- **Description**: These plots show residuals against predicted SFR values (ranging from -4 to 1) with a red dashed line at 0. Residuals are mostly random around zero, with some spread (up to 6 in Image 9) and a possible slight funnel shape at the extremes.
- **Interpretation**:
    - **Model Fit**: The random scatter around zero indicates no strong systematic bias, a positive sign for both models' validity in our theory verification. However, the slight widening of residuals at low and high predicted values (e.g., Image 9) hints at heteroscedasticity, an issue we flagged for further investigation (e.g., using weighted regression or transformations).
    - **Outliers**: Extreme residuals (e.g., 5-6 in Image 9) reinforce the need for outlier handling we discussed, such as filtering or robust methods, to enhance model reliability.

---

5. Distributions of SFR and Logmass

- **Relevant Images**: 10 ("SFR Distribution"), 11 ("Logmass Distribution")
- **Description**: The SFR histogram (Image 10) is right-skewed, peaking near 0 (frequency >1200) with a tail to -5. The logmass histogram (Image 11) peaks at 10-10.5 (frequency ~1400), with a skew from 6 to 12.
- **Interpretation**:
    - **Data Characteristics**: The skewed SFR distribution explains the models' struggles with extreme values—few galaxies have very low or high SFRs, limiting training data in those ranges. Similarly, the logmass peak at 10-10.5 suggests most galaxies are in a specific mass range, influencing feature importance (e.g., `log_Mass_gas`).
    - **Implications**: These distributions support our discussion on feature engineering (e.g., transformations to handle skewness) and collecting more diverse data to improve predictions at the extremes.

Overall Conclusion

The visuals demonstrate that both Random Forest and Gradient Boosting models effectively predict SFR, with Gradient Boosting slightly outperforming Random Forest due to its handling of nonlinear relationships (evident in feature importance and residual symmetry). Key findings align with our discussion:

- **Strengths**: High accuracy (scatter plots), minimal bias (residual histograms), and the pivotal role of gas mass (`log_Mass_gas`) in SFR prediction validate our theoretical framework.
- **Areas for Improvement**: Outliers, slight residual skewness, and potential heteroscedasticity suggest refining the models through:
    - **Outlier Handling**: Filter or adjust extreme values.
    - **Feature Engineering**: Add interactions (e.g., `log_Mass_gas * nsa_mstar`) and nonlinear terms.
    - **Model Tuning**: Address heteroscedasticity (e.g., via transformations) and enhance training data for extreme SFRs.

These steps will strengthen the models' predictive power and further verify our theory linking galaxy properties to star formation

Test 5 GB Model

Below is a complete, clear, and concise script for a Gradient Boosting (GB) model using "merged_data.csv" as the data source. This script focuses on the next steps you outlined: **tuning hyperparameters**, **checking residuals**, **validating externally**, and **adding more data** if needed. It also incorporates `OH_Mar13_N2_Re_fit` and `Av_gas_Re` to capture metallicity and dust effects, as well as interaction terms (e.g., `log_Mass_gas * nsa_mstar`) and nonlinear terms (e.g., `log_Mass_gas^2`) to enhance the model fit, given the dominance of `log_Mass_gas`. The approach builds on your strong results (e.g., test MSE of 0.2457, R^2 of 0.8438) to ensure reliable star formation rate (SFR) predictions.

Gradient Boosting Script

```
python
import pandas as pd
import numpy as np
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt

# --- 1. Data Preparation ---
# Load the dataset
df = pd.read_csv('merged_data.csv')
print(f"Dataset loaded with {len(df)} rows and {len(df.columns)} columns.")

# Replace sentinel values (e.g., -9999) with NaN
df = df.replace(-9999, np.nan)

# Define target and features
target = 'log_SFR_Ha'
base_features = ['log_Mass_gas', 'nsa_mstar', 'log_Mass', 'V-band_SB_at_Re',
                 'vel_sigma_Re', 'OH_Mar13_N2_Re_fit', 'Av_gas_Re']

# Impute missing values with median for robustness
```

```
imputer = SimpleImputer(strategy='median')
df[base_features] = imputer.fit_transform(df[base_features])

# Drop rows with missing target
df = df.dropna(subset=[target])
y = df[target]
X = df[base_features]

# --- 2. Feature Engineering ---
# Add interaction and nonlinear terms
df['log_Mass_gas_times_nsa_mstar'] = df['log_Mass_gas'] * df['nsa_mstar']
df['log_Mass_gas_times_metallicity'] = df['log_Mass_gas'] * df['OH_Mar13_N2_Re_fit']
df['log_Mass_gas_squared'] = df['log_Mass_gas'] ** 2

# Use PolynomialFeatures for systematic generation
poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly.fit_transform(X)
feature_names = poly.get_feature_names_out(base_features)

# --- 3. Model Training and Hyperparameter Tuning ---
# Split the data
X_train, X_test, y_train, y_test = train_test_split(X_poly, y, test_size=0.2, random_state=42)
print(f"Training set: {X_train.shape[0]} rows, Testing set: {X_test.shape[0]} rows")

# Define hyperparameter grid
param_grid = {
    'learning_rate': [0.01, 0.05, 0.1],
    'max_depth': [3, 5, 7],
    'n_estimators': [100, 200, 300]
}

# Initialize and tune Gradient Boosting model
gb = GradientBoostingRegressor(random_state=42)
grid_search = GridSearchCV(gb, param_grid, cv=5, scoring='neg_mean_squared_error', n_jobs=-1)
grid_search.fit(X_train, y_train)

# Get the best model
best_model = grid_search.best_estimator_
print(f"Best hyperparameters: {grid_search.best_params_}")

# --- 4. Model Evaluation ---
# Predict on test set
y_pred = best_model.predict(X_test)

# Calculate MSE and R-squared
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f"Test MSE: {mse:.4f}")
print(f"Test R-squared: {r2:.4f}")
```

```
# Feature importance
importances = best_model.feature_importances_
importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': importances})
print("\nTop 10 Important Features:")
print(importance_df.sort_values(by='Importance', ascending=False).head(10))

# Check residuals
residuals = y_test - y_pred
plt.figure(figsize=(8, 6))
plt.scatter(y_pred, residuals, alpha=0.5)
plt.axhline(0, color='red', linestyle='--')
plt.xlabel('Predicted log_SFR_Ha')
plt.ylabel('Residuals')
plt.title('Residuals vs. Predicted log_SFR_Ha')
plt.savefig('residuals.png')
plt.close()
print("Residual plot saved as 'residuals.png'. Inspect for patterns or biases.")

# --- 5. External Validation (Optional) ---
# Uncomment and adjust if you have an external dataset
"""
df_external = pd.read_csv('external_data.csv')
df_external[base_features] = imputer.transform(df_external[base_features])
X_external = poly.transform(df_external[base_features])
y_external = df_external['log_SFR_Ha']
y_external_pred = best_model.predict(X_external)
mse_external = mean_squared_error(y_external, y_external_pred)
r2_external = r2_score(y_external, y_external_pred)
print(f"External MSE: {mse_external:.4f}, R-squared: {r2_external:.4f}")
"""

# --- 6. Adding More Data (Optional) ---
# Uncomment and adjust if you want to merge additional data
"""
df_additional = pd.read_csv('mangaHIall.csv')
df_merged = pd.merge(df, df_additional, on='mangaid', how='left')
# Repeat feature engineering and retraining steps with df_merged
"""
```

Explanation of Key Steps

1. Data Preparation

- Loads "merged_data.csv" and replaces sentinel values (e.g., -9999) with NaN.
- Uses median imputation for missing values in features like `log_Mass_gas`, `OH_Mar13_N2_Re_fit`, and `Av_gas_Re`.
- Ensures the target (`log_SFR_Ha`) has no missing values.

2. Feature Engineering

- Includes base features plus `OH_Mar13_N2_Re_fit` (metallicity) and `Av_gas_Re` (dust effects) to improve SFR predictions.
- Adds interaction terms (e.g., `log_Mass_gas * nsa_mstar`) and nonlinear terms (e.g., `log_Mass_gas^2`) to capture complex relationships, leveraging `log_Mass_gas`'s dominance.
- Uses `PolynomialFeatures` for systematic feature generation.

3. Tuning Hyperparameters

- Splits data into 80% training and 20% testing sets.
- Uses `GridSearchCV` to tune `learning_rate`, `max_depth`, and `n_estimators`, aiming to lower MSE beyond your current 0.2457.

4. Checking Residuals

- Evaluates the model with MSE and R^2 on the test set.
- Plots residuals vs. predicted SFR and saves the plot as 'residuals.png'. Inspect this to identify if the model struggles with specific galaxies (e.g., systematic over- or under-predictions).

5. Validating Externally

- Provides a commented-out section for testing on an external dataset. You can uncomment and adjust it to validate generalizability beyond "merged_data.csv".

6. Adding More Data

- Includes a commented-out section for merging additional data (e.g., from "mangaHIall.csv"). Use this if performance plateaus, then repeat the process.

Let’s dive into the results of your Gradient Boosting (GB) model and use them to guide the next steps for refining and validating your Random Forest (RF) model, as requested. We’ll interpret the GB results, analyze the visuals, and then provide a script for the RF model that incorporates hyperparameter tuning, residual checking, external validation, and the option to add more data. This approach builds on your strong results while keeping things straightforward and reliable.

Interpreting the Gradient Boosting Results

Model Performance

- **Dataset**: The GB model was trained on "merged_data.csv" with 11,783 rows and 578 columns, split into 8,871 training rows (75%) and 2,218 testing rows (25%). This is a slightly different split from your previous runs (80-20), but the dataset size aligns with expectations.
- **Hyperparameters**: The best settings found were `learning_rate=0.05`, `max_depth=7`, and `n_estimators=100`. These suggest a moderately paced learning process with deeper trees to capture complex patterns, but not too many trees, balancing computation and performance.
- **Metrics**:
    - **Test MSE**: 0.2438, a slight improvement over your previous GB run (0.2457), indicating better prediction accuracy.
    - **Test R-squared**: 0.8513, also improved from 0.8438, meaning the model now explains 85.13% of the variance in star formation rates (SFR), a strong result for regression tasks.

Feature Importance

The top 10 features by importance are:

Rank	Feature	Importance
1	log_Mass_gas	0.365013
2	log_Mass_gas^2	0.253296
3	log_Mass_gas OH_Mar13_N2_Re_fit	0.178708
4	log_Mass_gas log_Mass	0.038425

5	log_Mass^2	0.034024
6	nsa_mstar log_Mass	0.026317
7	log_Mass	0.010321
8	nsa_mstar V-band_SB_at_Re	0.007671
9	V-band_SB_at_Re vel_sigma_Re	0.006681
10	V-band_SB_at_Re OH_Mar13_N2_Re_fit	0.005846

- **Dominance of Gas Mass**: `log_Mass_gas` and its nonlinear term (`log_Mass_gas^2`) lead, with a combined importance of ~0.62, reinforcing our discussion that gas mass is a key driver of SFR. The interaction with metallicity (`log_Mass_gas OH_Mar13_N2_Re_fit`) at 0.1787 shows that metallicity significantly modulates the gas-SFR relationship, as expected from astrophysical principles.
- **Other Features**: Stellar mass (`log_Mass`) and its interactions (e.g., with `nsa_mstar`) play a role, but their importance is lower, suggesting secondary effects compared to gas properties.

Visuals Analysis

- **Predicted vs. Actual log_SFR_Ha (Images 13, 14, 17)**:
    - The scatter plots show predicted vs. actual SFR values along a 1:1 line, with most points clustering tightly around it between -2 and 1. This indicates high accuracy, consistent with the R^2 of 0.8513. However, there's noticeable scatter at the extremes (<-3 and >1), suggesting the model struggles with very low or high SFR galaxies, a challenge we've noted before.
- **Histogram of Residuals (Images 12, 15)**:
    - The residuals (actual - predicted) are centered around 0, with a peak frequency of ~400-500. The distribution is fairly symmetric but has tails extending to ±2, indicating some prediction errors. Compared to previous runs (e.g., tails to 6), this is an improvement, but outliers remain.
- **Residuals vs. Predicted (Image 16)**:
    - The residuals vs. predicted plot shows a random scatter around zero, which is good—no systematic bias. However, there's a slight widening at the extremes (e.g., predicted SFR <-2 or >0), hinting at heteroscedasticity (uneven error variance), a point we've flagged for further investigation.

Random Forest Script for Validation

This script mirrors the GB approach, focusing on hyperparameter tuning, residual checking, external validation, and the option to add more data. It uses "merged_data.csv" and includes the same features and transformations.

```
python
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt

# --- 1. Data Preparation ---
# Load the dataset
df = pd.read_csv('merged_data.csv')
print(f"Dataset loaded with {len(df)} rows and {len(df.columns)} columns.")

# Replace sentinel values (e.g., -9999) with NaN
df = df.replace(-9999, np.nan)

# Define target and features
target = 'log_SFR_Ha'
base_features = ['log_Mass_gas', 'nsa_mstar', 'log_Mass', 'V-band_SB_at_Re',
                 'vel_sigma_Re', 'OH_Mar13_N2_Re_fit', 'Av_gas_Re']

# Impute missing values with median for robustness
imputer = SimpleImputer(strategy='median')
df[base_features] = imputer.fit_transform(df[base_features])

# Drop rows with missing target
df = df.dropna(subset=[target])
y = df[target]
X = df[base_features]

# --- 2. Feature Engineering ---
# Add interaction and nonlinear terms
df['log_Mass_gas_times_nsa_mstar'] = df['log_Mass_gas'] * df['nsa_mstar']
df['log_Mass_gas_times_metallicity'] = df['log_Mass_gas'] * df['OH_Mar13_N2_Re_fit']
df['log_Mass_gas_squared'] = df['log_Mass_gas'] ** 2

# Use PolynomialFeatures for systematic generation
poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly.fit_transform(X)
feature_names = poly.get_feature_names_out(base_features)
```

```
# --- 3. Model Training and Hyperparameter Tuning ---
# Split the data (75-25 split to match GB)
X_train, X_test, y_train, y_test = train_test_split(X_poly, y, test_size=0.25, random_state=42)
print(f"Training set: {X_train.shape[0]} rows, Testing set: {X_test.shape[0]} rows")

# Define hyperparameter grid
param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [10, 20, None],
    'min_samples_split': [2, 5, 10]
}

# Initialize and tune Random Forest model
rf = RandomForestRegressor(random_state=42)
grid_search = GridSearchCV(rf, param_grid, cv=5, scoring='neg_mean_squared_error', n_jobs=-1)
grid_search.fit(X_train, y_train)

# Get the best model
best_model = grid_search.best_estimator_
print(f"Best hyperparameters: {grid_search.best_params_}")

# --- 4. Model Evaluation ---
# Predict on test set
y_pred = best_model.predict(X_test)

# Calculate MSE and R-squared
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f"Test MSE: {mse:.4f}")
print(f"Test R-squared: {r2:.4f}")

# Feature importance
importances = best_model.feature_importances_
importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': importances})
print("\nTop 10 Important Features:")
print(importance_df.sort_values(by='Importance', ascending=False).head(10))

# Check residuals
residuals = y_test - y_pred
plt.figure(figsize=(8, 6))
plt.scatter(y_pred, residuals, alpha=0.5)
plt.axhline(0, color='red', linestyle='--')
plt.xlabel('Predicted log_SFR_Ha')
plt.ylabel('Residuals')
plt.title('Residuals vs. Predicted log_SFR_Ha (Random Forest)')
plt.savefig('residuals_rf.png')
plt.close()
print("Residual plot saved as 'residuals_rf.png'. Inspect for patterns or biases.")
```

```
# Residuals by SFR range
df_test = pd.DataFrame({'Predicted': y_pred, 'Residual': residuals, 'Actual': y_test})
df_test['SFR_Range'] = pd.cut(df_test['Actual'], bins=[-float('inf'), -2, 0, 2, float('inf')],
                              labels=['Very Low (<-2)', 'Low (-2 to 0)', 'High (0 to 2)', 'Very High (>2)'])
residual_stats = df_test.groupby('SFR_Range')['Residual'].agg(['mean', 'std']).reset_index()
print("\nResidual Statistics by SFR Range:")
print(residual_stats)

# --- 5. External Validation (Optional) ---
# Uncomment and adjust if you have an external dataset
"""
df_external = pd.read_csv('external_data.csv')
df_external[base_features] = imputer.transform(df_external[base_features])
X_external = poly.transform(df_external[base_features])
y_external = df_external['log_SFR_Ha']
y_external_pred = best_model.predict(X_external)
mse_external = mean_squared_error(y_external, y_external_pred)
r2_external = r2_score(y_external, y_external_pred)
print(f"External MSE: {mse_external:.4f}, R-squared: {r2_external:.4f}")
"""

# --- 6. Adding More Data (Optional) ---
# Uncomment and adjust if you want to merge additional data
"""
df_additional = pd.read_csv('mangaHIall.csv')
df_merged = pd.merge(df, df_additional, on='mangaid', how='left')
# Repeat feature engineering and retraining steps with df_merged
"""
```

How to Use the Script

1. **Save the Script**: Copy the code into a file named `rf_model_validation.py`.
2. **Prepare the Data**: Ensure "merged_data.csv" is in the same directory.
3. **Run the Script**: Execute `python rf_model_validation.py` in your terminal.
4. **Check Outputs**:
    - Console output will show MSE, R^2, feature importance, and residual statistics by SFR range.
    - A residual plot will be saved as `residuals_rf.png`. Inspect this for patterns (e.g., systematic errors or heteroscedasticity).
5. **Optional Steps**:
    - For external validation, uncomment and customize the "External Validation" section with your external dataset.

- To add more data, uncomment and adjust the "Adding More Data" section to merge with "mangaHIall.csv" or "Gema2.csv".

What the Script Does

1. Data Preparation

- Loads "merged_data.csv" and handles missing values using median imputation for features like `log_Mass_gas`, `OH_Mar13_N2_Re_fit`, and `Av_gas_Re`.
- Ensures the target (`log_SFR_Ha`) has no missing values.

2. Feature Engineering

- Includes base features, plus `OH_Mar13_N2_Re_fit` and `Av_gas_Re` to capture metallicity and dust effects.
- Adds interaction terms (e.g., `log_Mass_gas * nsa_mstar`) and nonlinear terms (e.g., `log_Mass_gas^2`) to leverage `log_Mass_gas`'s dominance.
- Uses `PolynomialFeatures` for systematic feature generation.

3. Hyperparameter Tuning

- Splits data into 75% training and 25% testing sets to match the GB run.
- Tunes `n_estimators`, `max_depth`, and `min_samples_split` using `GridSearchCV` to optimize performance.

4. Model Evaluation and Residual Checking

- Evaluates the model with MSE and R^2 on the test set.
- Lists the top 10 features by importance.
- Plots residuals vs. predicted SFR and saves as `residuals_rf.png`.
- Analyzes residuals by SFR range (very low, low, high, very high) to identify where the model struggles (e.g., extreme SFR values).

5. External Validation and Adding More Data

- Includes optional sections for external validation and merging additional data (e.g., from "mangaHIall.csv"), mirroring the GB script's structure.

Expected Outcomes

- **Performance**: Your previous RF run had an MSE of 0.2742 and R^2 of 0.8257. With hyperparameter tuning and the same feature set as the GB model, you might see improvements closer to the GB's MSE of 0.2438 and R^2 of 0.8513.
- **Residuals**: The residual analysis by SFR range will help identify if the model struggles with specific galaxies (e.g., very low or high SFRs), guiding further refinements.
- **Validation**: External validation will confirm if the model generalizes well, and adding more data can be explored if performance plateaus.

This script provides a robust way to test and validate your RF model, building on the GB model's success while keeping the process simple and focused. Let me know if you need help interpreting the results!

Results

(myenv) root@BigsbyGaming:/home/pmqr7# python rf_model.py

Dataset loaded with 11783 rows and 578 columns.
Training set: 8871 rows, Testing set: 2218 rows
Best hyperparameters: {'max_depth': 10, 'min_samples_split': 10, 'n_estimators': 200}
Test MSE: 0.2526
Test R-squared: 0.8459

Top 10 Important Features:

	Feature	Importance
7	log_Mass_gas^2	0.392928
0	log_Mass_gas	0.346726
12	log_Mass_gas OH_Mar13_N2_Re_fit	0.070701
9	log_Mass_gas log_Mass	0.027333

```
2                    log_Mass    0.025977
15            nsa_mstar log_Mass    0.025032
20                 log_Mass^2    0.021683
16       nsa_mstar V-band_SB_at_Re    0.006675
23     log_Mass OH_Mar13_N2_Re_fit    0.006199
29               vel_sigma_Re^2    0.005293
Residual plot saved as 'residuals_rf.png'. Inspect for patterns or biases.
```

Interpretation of the Most Recent Image: Residuals vs. Predicted log_SFR_Ha (Random Forest)

The most recent image provided is a scatter plot titled **"Residuals vs. Predicted log_SFR_Ha (Random Forest)"**, which serves as a residual plot generated from the Random Forest (RF) model described in your query. This plot is saved as `'residuals_rf.png'` and is intended to help assess the model's performance by visualizing the residuals (actual minus predicted values of the logarithm of the star formation rate, log_SFR_Ha) against the predicted values. Below is a detailed interpretation of this plot in the context of your RF model cross-validation results.

Description of the Plot

- **Axes**:
  - **X-axis (Predicted log_SFR_Ha)**: Ranges from -5 to 1, representing the predicted values of log_SFR_Ha output by the RF model. These values are logarithmically transformed star formation rates based on H-alpha measurements.
  - **Y-axis (Residuals)**: Ranges from -2 to 5, showing the residuals, calculated as the actual log_SFR_Ha minus the predicted log_SFR_Ha. A residual of zero indicates a perfect prediction.
- **Data Points**: The plot features numerous blue circular data points, with a high density clustered around predicted values between -2 and 0. Fewer points appear at the extremes (e.g., below -4 or above 0), and some outliers extend to residuals as high as 5 or as low as -2.
- **Reference Line**: A red dashed line at y = 0 represents the ideal scenario where residuals are zero (perfect predictions). Most points are scattered around this line, indicating the model's general accuracy.

Key Observations

1. **Residual Distribution**:
   - The majority of residuals are centered around zero, particularly for predicted values between -2 and 0, where the data points are most densely packed.

This suggests that the RF model accurately predicts log_SFR_Ha for the bulk of the dataset.
    - The symmetry of residuals around the zero line indicates that the model is unbiased overall—there is no consistent tendency to overpredict or underpredict across the range of predictions.
2. **Spread and Symmetry**:
    - The residuals are roughly symmetrically distributed around zero, with a slight widening of the spread at the lower end of predicted values (e.g., below -3). This could hint at mild heteroscedasticity (non-constant variance), where prediction errors increase for lower SFRs, but the effect is not severe.
    - For predicted values above 0, the residuals remain sparse but do not show a pronounced trend, maintaining the overall random scatter.
3. **Outliers**:
    - A few points exhibit large residuals, such as those reaching up to 5 (e.g., at predicted values around -4) or down to -2. These outliers suggest specific instances where the model significantly underperformed, either underestimating or overestimating the actual log_SFR_Ha.

Implications for Model Performance

- **Strengths**:
    - The random scatter of residuals around the zero line aligns with the reported **Test R^2 of 0.8459**, indicating that the model explains approximately 84.59% of the variance in log_SFR_Ha. This high R^2 reflects a strong predictive capability, corroborated by the tight clustering of residuals near zero for most predictions.
    - The **Test MSE of 0.2526** (mean squared error) is relatively low, and the residual plot supports this by showing that most errors are small, with only a few exceptions.
- **Challenges**:
    - The slight increase in residual spread at extreme predicted values (e.g., below -3) suggests that the model may struggle with very low SFR predictions. This could be due to limited data representation at these extremes or unmodeled complexities in the relationship between features and SFR.
    - The presence of outliers (e.g., residuals up to 5) indicates specific cases where the model's predictions deviate significantly from the actual values, potentially affecting its reliability for certain galaxies or conditions.

Connection to Feature Importance

Your query lists the **top 10 important features**, with `log_Mass_gas^2` (importance: 0.392928) and `log_Mass_gas` (0.346726) dominating the model, together accounting for over 73% of the feature importance. Other notable features include interaction terms like `log_Mass_gas OH_Mar13_N2_Re_fit` (0.070701) and `log_Mass_gas log_Mass` (0.027333).

- The residual plot suggests that the model effectively leverages these key features, particularly gas mass, to predict log_SFR_Ha accurately for most data points, as evidenced by the concentration of residuals near zero.
- However, the outliers may indicate instances where these dominant features alone do not fully capture the variability in SFR, possibly due to rare combinations of feature values or missing interactions not included in the model.

Model Context and Hyperparameters

- The RF model was trained on a dataset with **11,783 rows and 578 columns**, split into a training set of **8,871 rows** and a testing set of **2,218 rows**.
- The **best hyperparameters** identified via cross-validation are `max_depth=10`, `min_samples_split=10`, and `n_estimators=200`. These settings balance model complexity and generalization, contributing to the robust performance seen in the residual plot.

Recommendations for Improvement

Based on the residual plot and model metrics, here are some actionable steps to refine the RF model:

1. **Investigate Outliers**:
    - Examine data points with large residuals (e.g., > 3 or < -2) to determine if they result from data errors (e.g., measurement inaccuracies) or represent unique galaxy properties not captured by the current features. This could involve checking the corresponding values of `log_Mass_gas` and other key predictors.
2. **Address Potential Heteroscedasticity**:
    - The slight widening of residuals at lower predicted values suggests a minor variance issue. Consider applying transformations to the target variable (e.g., a different logarithmic scale) or using a weighted regression approach to stabilize the variance across the prediction range.
3. **Enhance Feature Engineering**:
    - Given the dominance of gas mass-related features, adding more interaction terms (e.g., `log_Mass_gas * OH_Mar13_N2_Re_fit`) or polynomial terms for

other variables might improve predictions, especially for outliers and extreme SFRs.

4. **Cross-Validation Consistency**:
    - Verify that the residual patterns observed here are consistent across all cross-validation folds to ensure the model's stability and generalizability.

Final Answer

The residual plot titled **"Residuals vs. Predicted log_SFR_Ha (Random Forest)"** indicates that your Random Forest model performs well overall, with most residuals centered around zero, supporting the strong performance metrics of **Test MSE: 0.2526** and **Test R^2: 0.8459**. This suggests that the model accurately predicts log_SFR_Ha for the majority of the test set (2,218 rows), effectively leveraging key features like `log_Mass_gas` and its squared term. However, the slight increase in residual spread at lower predicted values (e.g., below -3) and the presence of outliers (e.g., residuals up to 5) highlight areas for improvement, particularly in handling very low SFR predictions. Investigating these outliers and addressing potential heteroscedasticity could further enhance the model's reliability, ensuring robust SFR predictions across the entire range of your dataset (11,783 rows, 578 columns)

Suggested Approach to Continue Testing

1. Data Validation and Exploration

- **Verify Data Integrity**: Ensure all columns are correctly populated and free of anomalies (e.g., NaN values, extreme outliers).
- **Feature Analysis**: Explore correlations between features (e.g., `log_Mass_gas` vs. `log_SFR_Ha`) and check distributions to identify potential preprocessing needs (e.g., log transformations if not already applied).
- **Target Consistency**: Since you have two SFR indicators (`log_SFR_Ha` and `SFR_PETRORAD_R`), decide which to use as the target. `log_SFR_Ha` is likely your primary target due to its alignment with previous discussions, but you could compare predictions against `SFR_PETRORAD_R` for validation.

2. Feature Engineering

- **Enhance Features**: Build on existing interactions (e.g., `log_Mass_gas * nsa_mstar`) and add new ones involving `zcomp_1Mpc` (e.g., redshift-based density) or `LOGMSTARS` (stellar mass).
- **Polynomial Terms**: Include squared or higher-order terms for key predictors like `log_Mass_gas` to capture nonlinear relationships.

3. Model Training and Tuning

- **Train Both Models**: Use Random Forest and Gradient Boosting with hyperparameter tuning to optimize performance.
- **Cross-Validation**: Implement k-fold cross-validation to assess model stability and generalization.

4. Outlier Handling

- **Detect Outliers**: Identify galaxies with extreme `log_SFR_Ha` values using IQR or residual thresholds.
- **Evaluate Separately**: Report performance metrics for outliers and non-outliers to focus on robustness.

5. Validation and Visualization

- **External Validation**: If possible, test on a subset or separate dataset (e.g., from MaNGA).
- **Residual Analysis**: Plot residuals vs. predicted values to diagnose model fit.

6. Iterative Refinement

- Adjust features, hyperparameters, or outlier handling based on results, iterating until performance stabilizes.

Complete Testing Script

Here's a script to implement this approach:

```
python
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.model_selection import train_test_split, GridSearchCV, KFold
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt

# --- 1. Data Preparation ---
# Load dataset
df = pd.read_csv('complete_galaxy_dataset.csv')  # Assuming this is your merged file
print(f"Dataset loaded with {len(df)} rows and {len(df.columns)} columns.")
```

```
# Replace sentinel values with NaN
df = df.replace(-9999, np.nan)

# Define target and features
target = 'log_SFR_Ha'
features = ['log_Mass_gas', 'nsa_mstar', 'log_Mass', 'vel_sigma_Re',
            'OH_Mar13_N2_Re_fit', 'Av_gas_Re', 'zcomp_1Mpc', 'LOGMSTARS', 'SFR_PETRORAD_R']

# Impute missing values with median
imputer = SimpleImputer(strategy='median')
df[features] = imputer.fit_transform(df[features])

# Drop rows with missing target
df = df.dropna(subset=[target])
y = df[target]
X = df[features]

# --- 2. Feature Engineering ---
# Add interaction and nonlinear terms
df['log_Mass_gas_times_nsa_mstar'] = df['log_Mass_gas'] * df['nsa_mstar']
df['log_Mass_gas_times_metallicity'] = df['log_Mass_gas'] * df['OH_Mar13_N2_Re_fit']
df['log_Mass_gas_squared'] = df['log_Mass_gas'] ** 2
df['log_Mass_gas_times_zcomp'] = df['log_Mass_gas'] * df['zcomp_1Mpc']

# Generate polynomial features
poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly.fit_transform(X)
feature_names = poly.get_feature_names_out(features)

# --- 3. Outlier Detection ---
# Identify outliers using IQR on target
Q1, Q3 = y.quantile(0.25), y.quantile(0.75)
IQR = Q3 - Q1
outliers = (y < Q1 - 1.5 * IQR) | (y > Q3 + 1.5 * IQR)
print(f"Number of outliers in target: {outliers.sum()}")

# --- 4. Model Training and Hyperparameter Tuning ---
# Split the data
X_train, X_test, y_train, y_test = train_test_split(X_poly, y, test_size=0.2, random_state=42)
print(f"Training set: {X_train.shape[0]} rows, Testing set: {X_test.shape[0]} rows")

# Random Forest hyperparameter grid
rf_params = {
    'n_estimators': [100, 200],
    'max_depth': [10, 20],
    'min_samples_split': [5, 10]
}
rf = RandomForestRegressor(random_state=42)
rf_grid = GridSearchCV(rf, rf_params, cv=5, scoring='neg_mean_squared_error')
```

```
rf_grid.fit(X_train, y_train)
best_rf = rf_grid.best_estimator_
print(f"Best RF hyperparameters: {rf_grid.best_params_}")

# Gradient Boosting hyperparameter grid
gb_params = {
    'learning_rate': [0.01, 0.05],
    'max_depth': [3, 5],
    'n_estimators': [100, 200]
}
gb = GradientBoostingRegressor(random_state=42, loss='huber')
gb_grid = GridSearchCV(gb, gb_params, cv=5, scoring='neg_mean_squared_error')
gb_grid.fit(X_train, y_train)
best_gb = gb_grid.best_estimator_
print(f"Best GB hyperparameters: {gb_grid.best_params_}")

# --- 5. Model Evaluation ---
def evaluate_model(model, X_test, y_test, outliers, non_outliers):
    y_pred = model.predict(X_test)
    mse_full = mean_squared_error(y_test, y_pred)
    r2_full = r2_score(y_test, y_pred)
    mse_outliers = mean_squared_error(y_test[outliers], y_pred[outliers]) if outliers.sum() > 0 else np.nan
    r2_outliers = r2_score(y_test[outliers], y_pred[outliers]) if outliers.sum() > 0 else np.nan
    mse_non_outliers = mean_squared_error(y_test[non_outliers], y_pred[non_outliers])
    r2_non_outliers = r2_score(y_test[non_outliers], y_pred[non_outliers])
    return mse_full, r2_full, mse_outliers, r2_outliers, mse_non_outliers, r2_non_outliers

# Evaluate Random Forest
rf_results = evaluate_model(best_rf, X_test, y_test, outliers, ~outliers)
print("\nRandom Forest Results:")
print(f"Full Test Set - MSE: {rf_results[0]:.4f}, R-squared: {rf_results[1]:.4f}")
if not np.isnan(rf_results[2]):
    print(f"Outliers - MSE: {rf_results[2]:.4f}, R-squared: {rf_results[3]:.4f}")
print(f"Non-Outliers - MSE: {rf_results[4]:.4f}, R-squared: {rf_results[5]:.4f}")

# Evaluate Gradient Boosting
gb_results = evaluate_model(best_gb, X_test, y_test, outliers, ~outliers)
print("\nGradient Boosting Results:")
print(f"Full Test Set - MSE: {gb_results[0]:.4f}, R-squared: {gb_results[1]:.4f}")
if not np.isnan(gb_results[2]):
    print(f"Outliers - MSE: {gb_results[2]:.4f}, R-squared: {gb_results[3]:.4f}")
print(f"Non-Outliers - MSE: {gb_results[4]:.4f}, R-squared: {gb_results[5]:.4f}")

# --- 6. Visualization ---
# Residual plot for Random Forest
y_pred_rf = best_rf.predict(X_test)
residuals_rf = y_test - y_pred_rf
plt.figure(figsize=(8, 6))
plt.scatter(y_pred_rf, residuals_rf, alpha=0.5)
```

```
plt.axhline(0, color='red', linestyle='--')
plt.xlabel('Predicted log_SFR_Ha')
plt.ylabel('Residuals')
plt.title('Residuals vs. Predicted (Random Forest)')
plt.savefig('residuals_rf.png')
plt.close()

# Residual plot for Gradient Boosting
y_pred_gb = best_gb.predict(X_test)
residuals_gb = y_test - y_pred_gb
plt.figure(figsize=(8, 6))
plt.scatter(y_pred_gb, residuals_gb, alpha=0.5)
plt.axhline(0, color='red', linestyle='--')
plt.xlabel('Predicted log_SFR_Ha')
plt.ylabel('Residuals')
plt.title('Residuals vs. Predicted (Gradient Boosting)')
plt.savefig('residuals_gb.png')
plt.close()

# --- 7. Feature Importance ---
# Random Forest
rf_importance = pd.DataFrame({'Feature': feature_names, 'Importance': best_rf.feature_importances_})
print("\nTop 10 Important Features (Random Forest):")
print(rf_importance.sort_values(by='Importance', ascending=False).head(10))

# Gradient Boosting
gb_importance = pd.DataFrame({'Feature': feature_names, 'Importance': best_gb.feature_importances_})
print("\nTop 10 Important Features (Gradient Boosting):")
print(gb_importance.sort_values(by='Importance', ascending=False).head(10))
```

How to Use the Script

1. **Save the Script**: Copy the code into a file named `gbrf_sfr_test.py`.
2. **Prepare Your Data**: Ensure `complete_galaxy_dataset.csv` (or your merged file) is in the same directory, containing the specified columns.
3. **Run the Script**: Execute with `python gbrf_sfr_test.py`.
4. **Review Outputs**:
    - Console output includes dataset size, number of outliers, MSE and R^2 for full test set, outliers, and non-outliers, and feature importance.
    - Residual plots are saved as `residuals_rf.png` and `residuals_gb.png`.
5. **Customize (Optional)**:
    - Adjust `base_features` or `target` if your column names differ.
    - Modify hyperparameter grids (`rf_params`, `gb_params`) for more extensive tuning.

Key Features and Notes

- **Data Preparation**: Handles missing values and ensures a clean dataset.
- **Feature Engineering**: Includes interactions (e.g., `log_Mass_gas_times_zcomp`) and polynomial terms to capture complex relationships.
- **Outlier Detection**: Uses IQR to identify outliers in `log_SFR_Ha`, allowing separate evaluation.
- **Model Tuning**: Employs `GridSearchCV` with reasonable hyperparameter ranges to optimize performance.
- **Evaluation**: Provides detailed metrics for full, outlier, and non-outlier subsets.
- **Visualization**: Saves residual plots for diagnostic analysis.

This script provides a robust framework to continue testing your GBRF model with the new dataset, focusing on refinement and validation. Let me know if you encounter issues or need further adjustments!

this is the error,
(myenv) root@BigsbyGaming:/home/pmqr7# python GBRFSFR.py Dataset loaded with 11222 rows and 11 columns. Number of outliers in target: 248 Training set: 8977 rows, Testing set: 2245 rows Best RF hyperparameters: {'max_depth': 20, 'min_samples_split': 10, 'n_estimators': 300} Best GB hyperparameters: {'learning_rate': 0.05, 'max_depth': 5, 'n_estimators': 200} Traceback (most recent call last): File "/home/pmqr7/GBRFSFR.py", line 93, in <module> rf_results = evaluate_model(best_rf, X_test, y_test, outliers, ~outliers) ^^^

^^^

`X_train`, `X_test`, `y_train`, `y_test`. But the `outliers` array is based on the full `y`, so it's not directly applicable to `y_test`.
One solution is to identify outliers separately for the test set. But actually, since outliers were identified based on the entire dataset, it might be more consistent to use the outliers that correspond to the test set indices.
Wait, perhaps I can use the indices from the split to subset the `outliers` array.
Let me think about how `train_test_split` works. By default, it shuffles the data and splits it. So, the rows in `y_test` correspond to a subset of the original `y`.
To get the correct outliers for the test set, I need to know which indices were selected for the test set.
Actually, `train_test_split` can return the indices if we specify `return_indices=True`, but I don't think that's the case here. Wait, looking back at the script, it's probably just splitting the arrays without returning indices.
Alternatively, since `y_test` is a subset of `y`, I can find which elements of `y` are in `y_test`, but that might not be straightforward because `y` could have duplicate values.
Wait, a better way is to keep track of the indices when splitting.
Let me suggest modifying the data splitting part to also split the `outliers` array.
For example, when splitting, we can do:
`python`
X_train, X_test, y_train, y_test, outliers_train, outliers_test = train_test_split(X, y, outliers, test_size=0.2, random_state=42)

Wait, actually, `train_test_split` can take multiple arrays to split consistently. So, if `outliers` is an array corresponding to `y`, we can split it along with `X` and `y`.
Assuming `outliers` is a boolean array of the same length as `y`, we can do:
`python`
X_train, X_test, y_train, y_test, outliers_train, outliers_test = train_test_split(X_poly, y, outliers, test_size=0.2, random_state=42)

Then, in the `evaluate_model` function, use `outliers_test` instead of `outliers`.
Wait, actually, in the error message, it's using `outliers` for both the full dataset and the test set, which is incorrect.
So, to fix it, I need to pass `outliers_test` to the `evaluate_model` function.
Let me check the script again. In the script, it's calling:
`python`
rf_results = evaluate_model(best_rf, X_test, y_test, outliers, ~outliers)

But `outliers` is the full dataset's outliers, not the test set's.
So, yes, I need to split `outliers` into `outliers_train` and `outliers_test`.
Wait, but actually, in the `evaluate_model` function, it's using `outliers` to index `y_test`, which is only the test set.
So, to make it work, I need to pass the outliers corresponding to the test set.
Alternatively, since the outliers were identified based on the full dataset, perhaps I should identify outliers only on the test set.

But that might not be consistent, because the outlier detection was based on the entire dataset.
Wait, another way is to use the indices.
Actually, when we do `train_test_split`, it returns the subsets, but we can also get the indices.
Wait, by default, `train_test_split` returns the subsets, but we can get the indices if we want.
Alternatively, since `y_test` is a subset of `y`, and `outliers` is a boolean array for `y`, I can find which elements of `y` are in `y_test`.
But that might be tricky.
Wait, actually, since `y_test` is a pandas Series or numpy array, and `y` is the full array, I can use the indices.
Wait, if `y` is a pandas Series with an index, and `y_test` has the same index, then I can use the index to subset `outliers`.
But in the script, it's likely that `y` is a numpy array or a pandas Series without a specific index.
Wait, in the error message, it says "boolean index did not match indexed array along axis 0; size of axis is 2245 but size of corresponding boolean axis is 11222", which suggests that `y_test` is a numpy array of size 2245, and `outliers` is a boolean array of size 11222.
So, to fix this, I need a boolean array of size 2245 indicating which elements in `y_test` are outliers.
One way to do this is to identify the outliers based on the test set only, but that might not be what was intended.
Alternatively, since the outliers were identified based on the entire dataset, I can use the indices to map the outliers to the test set.
Wait, perhaps the best way is to keep the indices when splitting.
Let me suggest modifying the splitting part.
First, let's assume that `y` is a pandas Series with an index, or if it's a numpy array, we can create an index.
Actually, in the script, since it's loaded from a CSV, it's likely a pandas DataFrame, and `y` is a Series with an index.
Wait, looking back at the script, it's probably:
`python`

```
df = pd.read_csv('compiled_sfr_dataset.csv')
# ...
y = df[target]
# ...
X_train, X_test, y_train, y_test = train_test_split(X_poly, y, test_size=0.2, random_state=42)
```

So, `y` is a pandas Series with an index, and `y_test` is a subset with the same index.
Similarly, `outliers` is likely a boolean Series or array corresponding to `y`.
Wait, in the script, `outliers` is probably computed as:
`python`

Q1, Q3 = y.quantile(0.25), y.quantile(0.75)
IQR = Q3 - Q1
outliers = (y < Q1 - 1.5 * IQR) | (y > Q3 + 1.5 * IQR)

So, `outliers` is a boolean Series with the same index as `y`.
Therefore, when we have `y_test`, which is a subset of `y`, we can use the index to get the corresponding outliers.
Specifically, since `y_test` has an index, we can do:
`python`
outliers_test = outliers.loc[y_test.index]

Then, in the `evaluate_model` function, use `outliers_test` instead of `outliers`.
Similarly, for non-outliers, use `~outliers_test`.
So, in the script, after splitting the data, add:
`python`
outliers_test = outliers.loc[y_test.index]

Then, when calling `evaluate_model`, pass `outliers_test` and `~outliers_test`.
Wait, actually, in the script, it's passing `outliers` and `~outliers`, but since `outliers` is for the full dataset, I need to pass the subset.
Alternatively, to make it more general, I can modify the `evaluate_model` function to accept the full `outliers` and subset it inside the function.
But perhaps the simplest is to subset it before calling.
Let me suggest the following fix:
After the data splitting, add:
`python`
Assuming y is a pandas Series with an index
outliers_test = outliers.loc[y_test.index]

Then, when calling `evaluate_model`, use:
`python`
rf_results = evaluate_model(best_rf, X_test, y_test, outliers_test, ~outliers_test)

Similarly for GB.
Wait, and make sure that `outliers` is a pandas Series with the same index as `y`.
If `y` is a numpy array, then it's trickier.
Wait, in the script, since `y = df[target]`, and `df` is a DataFrame, `y` is a Series with an index.
And `outliers` is computed from `y`, so it should have the same index.
Wait, actually, when we do `(y < Q1 - 1.5 * IQR) | (y > Q3 + 1.5 * IQR)`, it returns a boolean Series with the same index as `y`.
So, yes, `outliers` has the same index as `y`.
Therefore, `outliers.loc[y_test.index]` will give the outliers for the test set.
So, this should work.
Alternatively, if for some reason `y_test` doesn't have an index (e.g., if it's converted to a numpy array), we might need to handle it differently.

But in standard usage, since `train_test_split` preserves the index when splitting pandas Series, `y_test` should have an index.
To confirm, you can add a print statement:
`python`
print(y_test.index)

If it shows an index, then `outliers.loc[y_test.index]` should work.
If not, we might need to use the positions.
But I think it should be fine.
So, to summarize, the fix is:
In the script, after the data splitting, add:
`python`
outliers_test = outliers.loc[y_test.index]

Then, in the calls to `evaluate_model`, use `outliers_test` and `~outliers_test`:
`python`
rf_results = evaluate_model(best_rf, X_test, y_test, outliers_test, ~outliers_test)
gb_results = evaluate_model(best_gb, X_test, y_test, outliers_test, ~outliers_test)

This should resolve the IndexError, as `outliers_test` will have the same length as `y_test`.
Also, make sure that `~outliers_test` is used correctly; since `outliers_test` is a boolean Series, `~outliers_test` will be the non-outliers.
Wait, actually, in the `evaluate_model` function, it's using `non_outliers`, so it should be fine.
Alternatively, to make it more robust, you can modify the `evaluate_model` function to accept the full `outliers` and subset it inside, but for now, this should suffice.
So, the next step is to edit the script to include the line:
`python`
outliers_test = outliers.loc[y_test.index]

after the data splitting, and then use `outliers_test` in the `evaluate_model` calls.
Then, run the script again, and it should proceed without the IndexError.
If there are further issues, we can address them accordingly.
But this should fix the current error.
Final Answer
To resolve the `IndexError` in your script, you need to ensure that the `outliers` boolean array matches the size of the `y_test` array when evaluating the model. The error occurs because the `outliers` array is based on the full dataset, while `y_test` is only a subset (the test set).

Solution

1. **Identify Outliers for the Test Set:**
    - Since `outliers` is a boolean array corresponding to the full dataset `y`, you need to subset it to match the test set indices.

    - Assuming `y` is a pandas Series with an index, you can extract the outliers for the test set using the index of `y_test`.
2. **Modify the Script:**
    - After splitting the data with `train_test_split`, add the following line to create a subset of outliers for the test set:
    - `python`
    - outliers_test = outliers.loc[y_test.index]
    - Then, when calling the `evaluate_model` function, use `outliers_test` instead of `outliers`:
    - `python`

rf_results = evaluate_model(best_rf, X_test, y_test, outliers_test, ~outliers_test)

    - gb_results = evaluate_model(best_gb, X_test, y_test, outliers_test, ~outliers_test)

Explanation

- The `outliers` array is created based on the full target variable `y` (11,222 elements), but `y_test` only contains 2,245 elements (the test set).
- By using `outliers.loc[y_test.index]`, you create a boolean array `outliers_test` that corresponds specifically to the test set, ensuring it has the same size as `y_test`.
- This allows you to correctly index `y_test` and `y_pred` with `outliers_test` and `~outliers_test` in the `evaluate_model` function.

Updated Script Snippet

Here's how the relevant part of your script should look:
`python`
```
# After data splitting
X_train, X_test, y_train, y_test = train_test_split(X_poly, y, test_size=0.2, random_state=42)
outliers_test = outliers.loc[y_test.index]  # Subset outliers for test set

# When evaluating models
rf_results = evaluate_model(best_rf, X_test, y_test, outliers_test, ~outliers_test)
gb_results = evaluate_model(best_gb, X_test, y_test, outliers_test, ~outliers_test)
```

By making this adjustment, the script will correctly evaluate the model's performance on the test set, including separate metrics for outliers and non-outliers, without encountering the `IndexError`.

You've reached the limit of 25 attachments in this conversation. Grok might forget content from earlier attachments; please start a new conversation to upload more attachments.

Understanding the Error

You're encountering an `IndexError` when running your `GBRFSFR.py` script:
IndexError: boolean index did not match indexed array along axis 0; size of axis is 2245 but size of corresponding boolean axis is 11222

This error occurs in the `evaluate_model` function at line 86:
`python`
mse_outliers = mean_squared_error(y_test[outliers], y_pred[outliers]) if outliers.sum() > 0 else np.nan

The issue is a mismatch in sizes:

- `y_test` has 2,245 elements (the size of your test set).
- `outliers` is a boolean array with 11,222 elements (the size of the entire dataset).

When you try to index `y_test` with `outliers`, Python raises an error because the boolean mask (`outliers`) must have the same length as the array it's indexing (`y_test`), but here it's much larger.

Why This Happens

From your script's output:

- The dataset has 11,222 rows.
- You identified 248 outliers in the target variable (`y`) before splitting the data.
- The data is split into a training set (8,977 rows) and a testing set (2,245 rows) using `train_test_split`.

The `outliers` array is created based on the full target variable `y` (11,222 elements). However, when evaluating the model, you're working with `y_test`, which is only the test portion (2,245 elements). Passing the full `outliers` array to index `y_test` causes the size mismatch.

How to Fix It

To resolve this, you need a boolean array that matches the size of `y_test` (2,245 elements) to indicate which test set elements are outliers. Since `outliers` corresponds to the full dataset and `y` is likely a pandas Series (given the dataset is loaded from a CSV), you can use the index of `y_test` to subset `outliers` for the test set.
Here's the step-by-step solution:

1. **After Splitting the Data:**
    - After you split your data with `train_test_split`, `y_test` retains the index from the original `y` (assuming `y` is a pandas Series).
    - Subset the `outliers` array to match `y_test`'s index.
2. Add this line after your data split:
3. `python`
4. outliers_test = outliers.loc[y_test.index]
5. **Update the Model Evaluation Calls:**
    - When calling `evaluate_model`, use `outliers_test` instead of `outliers`:
6. `python`

rf_results = evaluate_model(best_rf, X_test, y_test, outliers_test, ~outliers_test)

7. gb_results = evaluate_model(best_gb, X_test, y_test, outliers_test, ~outliers_test)

Updated Script Snippet

Assuming your script looks something like this:
`python`
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
import numpy as np
# ... (other imports)

# Load data
df = pd.read_csv('compiled_sfr_dataset.csv')
print(f"Dataset loaded with {df.shape[0]} rows and {df.shape[1]} columns.")
target = 'your_target_column'  # Replace with your target column name
y = df[target]
X = df.drop(columns=[target])
# ... (preprocessing, e.g., creating X_poly)

# Identify outliers
Q1, Q3 = y.quantile(0.25), y.quantile(0.75)
IQR = Q3 - Q1
outliers = (y < Q1 - 1.5 * IQR) | (y > Q3 + 1.5 * IQR)
print(f"Number of outliers in target: {outliers.sum()}")

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print(f"Training set: {X_train.shape[0]} rows, Testing set: {X_test.shape[0]} rows")

# ... (model training, hyperparameter tuning)
```

```
# Evaluate models
rf_results = evaluate_model(best_rf, X_test, y_test, outliers, ~outliers)
gb_results = evaluate_model(best_gb, X_test, y_test, outliers, ~outliers)
```

Modify it to:

```
python
# ... (previous code unchanged)

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print(f"Training set: {X_train.shape[0]} rows, Testing set: {X_test.shape[0]} rows")

# Subset outliers for the test set
outliers_test = outliers.loc[y_test.index]

# ... (model training, hyperparameter tuning)

# Evaluate models
rf_results = evaluate_model(best_rf, X_test, y_test, outliers_test, ~outliers_test)
gb_results = evaluate_model(best_gb, X_test, y_test, outliers_test, ~outliers_test)
```

Verification

- `outliers_test` will have the same length as `y_test` (2,245), because it's indexed using `y_test.index`.
- Since `outliers` is a boolean pandas Series with the same index as `y`, `outliers.loc[y_test.index]` correctly aligns the outliers with the test set.
- `~outliers_test` computes the non-outliers within the test set, also matching `y_test`'s size.

If `y` Is a NumPy Array

If `y` is a NumPy array (not a pandas Series), `y_test` won't have an index. In that case, you'd need to split `outliers` alongside `X` and `y`:

```
python
X_train, X_test, y_train, y_test, outliers_train, outliers_test = train_test_split(
    X, y, outliers, test_size=0.2, random_state=42
)
```

Then use `outliers_test` as above. However, since your dataset comes from a CSV and `y` is likely a pandas Series, the `.loc` approach should work.

Final Steps

1. Edit your `GBRFSFR.py` script with the changes above.
2. Run it again:
3. `bash`
4. python GBRFSFR.py
5. The `IndexError` should be resolved, and your script should complete the model evaluation.

If you encounter further issues, double-check that `y` is a pandas Series and that `outliers` has the same index. You can add `print(y_test.index)` after the split to confirm the index is preserved.

Below is a complete Python script for building and evaluating a Gradient Boosting and Random Forest Star Formation Rate (GBRFSFR) prediction model based on the recommendations provided. This script assumes your dataset is stored in a CSV file named `compiled_sfr_dataset.csv`. It includes data loading, preprocessing, outlier detection, model training with hyperparameter tuning, evaluation, cross-validation, visualization, and feature importance analysis.

`python`

```
import pandas as pd
import numpy as np
from sklearn.ensemble import GradientBoostingRegressor, RandomForestRegressor
from sklearn.model_selection import train_test_split, RandomizedSearchCV, KFold, cross_val_score
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt
import joblib

# --- 1. Data Preparation ---
# Load dataset
df = pd.read_csv('compiled_sfr_dataset.csv')
print(f"Dataset loaded with {df.shape[0]} rows and {df.shape[1]} columns.")

# Define target and features (adjust these column names as per your dataset)
target = 'log_SFR_Ha'
features = ['log_Mass_gas', 'nsa_mstar', 'log_Mass', 'vel_sigma_Re',
            'OH_Mar13_N2_Re_fit', 'Av_gas_Re', 'zcomp_1Mpc', 'LOGMSTARS', 'SFR_PETRORAD_R']
X = df[features]
y = df[target]

# Handle missing values by dropping rows with NaNs
X = X.dropna()
y = y.loc[X.index]  # Align y with X after dropping rows

# --- 2. Outlier Detection ---
```

```
# Identify outliers in the target variable using IQR
Q1, Q3 = y.quantile(0.25), y.quantile(0.75)
IQR = Q3 - Q1
outliers = (y < Q1 - 1.5 * IQR) | (y > Q3 + 1.5 * IQR)
print(f"Number of outliers in target: {outliers.sum()}")

# --- 3. Data Splitting ---
# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print(f"Training set: {X_train.shape[0]} rows, Testing set: {X_test.shape[0]} rows")

# Subset outliers for the test set
outliers_test = outliers.loc[y_test.index]

# --- 4. Model Training with Hyperparameter Tuning ---
# Define parameter grids for RandomizedSearchCV
rf_params = {
    'n_estimators': [100, 200, 300],
    'max_depth': [10, 20, None],
    'min_samples_split': [2, 5, 10]
}

gb_params = {
    'learning_rate': [0.01, 0.05, 0.1],
    'max_depth': [3, 5, 7],
    'n_estimators': [100, 200, 300]
}

# RandomizedSearchCV for Random Forest
rf = RandomForestRegressor(random_state=42)
rf_random = RandomizedSearchCV(rf, rf_params, n_iter=10, cv=3, scoring='neg_mean_squared_error',
n_jobs=-1, verbose=2)
rf_random.fit(X_train, y_train)
best_rf = rf_random.best_estimator_
print(f"Best RF hyperparameters: {rf_random.best_params_}")

# RandomizedSearchCV for Gradient Boosting
gb = GradientBoostingRegressor(random_state=42, loss='huber')
gb_random = RandomizedSearchCV(gb, gb_params, n_iter=10, cv=3,
scoring='neg_mean_squared_error', n_jobs=-1, verbose=2)
gb_random.fit(X_train, y_train)
best_gb = gb_random.best_estimator_
print(f"Best GB hyperparameters: {gb_random.best_params_}")

# --- 5. Model Evaluation ---
def evaluate_model(model, X_test, y_test, outliers_test):
    """Evaluate model performance on full test set, outliers, and non-outliers."""
    y_pred = model.predict(X_test)
    mse_full = mean_squared_error(y_test, y_pred)
```

```
    r2_full = r2_score(y_test, y_pred)
    if outliers_test.sum() > 0:
        mse_outliers = mean_squared_error(y_test[outliers_test], y_pred[outliers_test])
        r2_outliers = r2_score(y_test[outliers_test], y_pred[outliers_test])
    else:
        mse_outliers = np.nan
        r2_outliers = np.nan
    mse_non_outliers = mean_squared_error(y_test[~outliers_test], y_pred[~outliers_test])
    r2_non_outliers = r2_score(y_test[~outliers_test], y_pred[~outliers_test])
    return mse_full, r2_full, mse_outliers, r2_outliers, mse_non_outliers, r2_non_outliers

# Evaluate Random Forest
rf_results = evaluate_model(best_rf, X_test, y_test, outliers_test)
print("\nRandom Forest Results:")
print(f"Full Test Set - MSE: {rf_results[0]:.4f}, R-squared: {rf_results[1]:.4f}")
print(f"Outliers - MSE: {rf_results[2]:.4f}, R-squared: {rf_results[3]:.4f}")
print(f"Non-Outliers - MSE: {rf_results[4]:.4f}, R-squared: {rf_results[5]:.4f}")

# Evaluate Gradient Boosting
gb_results = evaluate_model(best_gb, X_test, y_test, outliers_test)
print("\nGradient Boosting Results:")
print(f"Full Test Set - MSE: {gb_results[0]:.4f}, R-squared: {gb_results[1]:.4f}")
print(f"Outliers - MSE: {gb_results[2]:.4f}, R-squared: {gb_results[3]:.4f}")
print(f"Non-Outliers - MSE: {gb_results[4]:.4f}, R-squared: {gb_results[5]:.4f}")

# --- 6. K-Fold Cross-Validation ---
kf = KFold(n_splits=5, shuffle=True, random_state=42)

# Random Forest cross-validation
rf_cv_mse = -cross_val_score(best_rf, X, y, cv=kf, scoring='neg_mean_squared_error').mean()
rf_cv_r2 = cross_val_score(best_rf, X, y, cv=kf, scoring='r2').mean()
print(f"\nRandom Forest K-Fold CV - Average MSE: {rf_cv_mse:.4f}, Average R-squared: {rf_cv_r2:.4f}")

# Gradient Boosting cross-validation
gb_cv_mse = -cross_val_score(best_gb, X, y, cv=kf, scoring='neg_mean_squared_error').mean()
gb_cv_r2 = cross_val_score(best_gb, X, y, cv=kf, scoring='r2').mean()
print(f"Gradient Boosting K-Fold CV - Average MSE: {gb_cv_mse:.4f}, Average R-squared:
{gb_cv_r2:.4f}")

# --- 7. Visualization ---
# Residual plot for Random Forest
y_pred_rf = best_rf.predict(X_test)
residuals_rf = y_test - y_pred_rf
plt.figure(figsize=(8, 6))
plt.scatter(y_pred_rf, residuals_rf, alpha=0.5)
plt.axhline(0, color='red', linestyle='--')
plt.xlabel('Predicted log_SFR_Ha')
plt.ylabel('Residuals')
plt.title('Residuals vs. Predicted (Random Forest)')
```

```
plt.savefig('residuals_rf.png')
plt.close()

# Residual plot for Gradient Boosting
y_pred_gb = best_gb.predict(X_test)
residuals_gb = y_test - y_pred_gb
plt.figure(figsize=(8, 6))
plt.scatter(y_pred_gb, residuals_gb, alpha=0.5)
plt.axhline(0, color='red', linestyle='--')
plt.xlabel('Predicted log_SFR_Ha')
plt.ylabel('Residuals')
plt.title('Residuals vs. Predicted (Gradient Boosting)')
plt.savefig('residuals_gb.png')
plt.close()

# --- 8. Feature Importance ---
# Random Forest feature importance
rf_importance = pd.DataFrame({'Feature': features, 'Importance': best_rf.feature_importances_})
print("\nTop 10 Important Features (Random Forest):")
print(rf_importance.sort_values(by='Importance', ascending=False).head(10))

# Gradient Boosting feature importance
gb_importance = pd.DataFrame({'Feature': features, 'Importance': best_gb.feature_importances_})
print("\nTop 10 Important Features (Gradient Boosting):")
print(gb_importance.sort_values(by='Importance', ascending=False).head(10))

# --- Optional: Save Models ---
# Uncomment the lines below to save the trained models
# joblib.dump(best_rf, 'best_rf_model.pkl')
# joblib.dump(best_gb, 'best_gb_model.pkl')
```

Explanation of the Script

1. Data Preparation

- **Loading**: Loads the dataset from `compiled_sfr_dataset.csv` and prints its dimensions.
- **Features and Target**: Defines the target variable (`log_SFR_Ha`) and a list of feature columns. Adjust these as needed to match your dataset.
- **Missing Values**: Drops rows with missing values in the features and aligns the target accordingly.

2. Outlier Detection

- Uses the Interquartile Range (IQR) method to identify outliers in the target variable (`log_SFR_Ha`) and prints the count.

3. Data Splitting

- Splits the data into 80% training and 20% testing sets with a fixed random state for reproducibility.
- Subsets the outliers for the test set using the test indices.

4. Model Training with Hyperparameter Tuning

- **Parameter Grids**: Defines hyperparameter ranges for Random Forest (`n_estimators`, `max_depth`, `min_samples_split`) and Gradient Boosting (`learning_rate`, `max_depth`, `n_estimators`).
- **RandomizedSearchCV**: Performs hyperparameter tuning with 10 iterations and 3-fold cross-validation, optimizing for negative MSE. Uses `n_jobs=-1` to utilize all CPU cores.
- **Best Models**: Fits the best Random Forest and Gradient Boosting models based on the tuning results.

5. Model Evaluation

- **Custom Function**: `evaluate_model` computes MSE and R-squared for the full test set, outliers, and non-outliers separately.
- **Results**: Prints evaluation metrics for both models, handling cases where no outliers exist in the test set.

6. K-Fold Cross-Validation

- Performs 5-fold cross-validation on the entire dataset to assess model generalization, reporting average MSE and R-squared.

7. Visualization

- Generates residual plots for both models, showing residuals (true - predicted) against predicted values. Saves plots as PNG files (`residuals_rf.png` and `residuals_gb.png`).

8. Feature Importance

- Extracts and displays the top 10 most important features for both models based on their `feature_importances_` attribute.

Optional: Model Saving

- Includes commented-out lines to save the trained models using `joblib` for later use.

Notes

- **Dataset Path**: Ensure `compiled_sfr_dataset.csv` is in the same directory as the script, or modify the path accordingly.
- **Column Names**: Adjust `target` and `features` to match your dataset's column names.
- **Feature Scaling**: Not included since tree-based models (Random Forest and Gradient Boosting) are insensitive to feature scaling. Add scaling (e.g., `StandardScaler`) before splitting if using other model types.
- **Feature Engineering**: Assumes features are preprocessed. Add interaction terms or nonlinear transformations before splitting if needed.
- **Error Handling**: The script assumes a clean dataset. Add checks for missing columns or data type issues if necessary.

This script provides a comprehensive workflow for training, evaluating, and interpreting the GBRFSFR prediction model.

(myenv) root@BigsbyGaming:/home/pmqr7# python GBRFSFR_2.py Dataset loaded with 11222 rows and 11 columns. Number of outliers in target: 248 Training set: 8977 rows, Testing set: 2245 rows Fitting 3 folds for each of 10 candidates, totalling 30 fits [CV] END max_depth=10, min_samples_split=10, n_estimators=100; total time= 4.2s [CV] END max_depth=10, min_samples_split=10, n_estimators=100; total time= 4.3s [CV] END max_depth=10, min_samples_split=10, n_estimators=100; total time= 4.3s [CV] END max_depth=10, min_samples_split=5, n_estimators=100; total time= 4.0s [CV] END max_depth=10, min_samples_split=5, n_estimators=100; total time= 4.0s [CV] END max_depth=10, min_samples_split=5, n_estimators=100; total time= 4.2s [CV] END max_depth=None, min_samples_split=10, n_estimators=200; total time= 10.2s [CV] END max_depth=None, min_samples_split=10, n_estimators=200; total time= 10.5s [CV] END max_depth=None, min_samples_split=10, n_estimators=200; total time= 10.5s [CV] END max_depth=None, min_samples_split=5, n_estimators=200; total time= 11.4s [CV] END max_depth=None, min_samples_split=5, n_estimators=200; total time= 11.5s [CV] END max_depth=None, min_samples_split=5, n_estimators=200; total time= 11.6s [CV] END max_depth=20, min_samples_split=10, n_estimators=100; total time= 5.4s [CV] END max_depth=20, min_samples_split=10, n_estimators=100; total time= 5.4s [CV] END max_depth=20, min_samples_split=10, n_estimators=100; total time= 5.7s [CV] END max_depth=20, min_samples_split=2, n_estimators=300; total time= 18.2s [CV] END max_depth=20, min_samples_split=2, n_estimators=300; total time= 19.5s [CV] END

max_depth=20, min_samples_split=2, n_estimators=300; total time= 20.0s [CV] END max_depth=10, min_samples_split=10, n_estimators=300; total time= 12.3s [CV] END max_depth=10, min_samples_split=10, n_estimators=300; total time= 12.6s [CV] END max_depth=10, min_samples_split=10, n_estimators=300; total time= 13.3s [CV] END max_depth=20, min_samples_split=5, n_estimators=100; total time= 5.4s [CV] END max_depth=20, min_samples_split=10, n_estimators=300; total time= 14.7s [CV] END max_depth=20, min_samples_split=5, n_estimators=100; total time= 5.1s [CV] END max_depth=20, min_samples_split=5, n_estimators=100; total time= 5.0s [CV] END max_depth=20, min_samples_split=10, n_estimators=300; total time= 14.8s [CV] END max_depth=20, min_samples_split=10, n_estimators=300; total time= 15.3s [CV] END max_depth=20, min_samples_split=5, n_estimators=300; total time= 13.4s [CV] END max_depth=20, min_samples_split=5, n_estimators=300; total time= 13.8s [CV] END max_depth=20, min_samples_split=5, n_estimators=300; total time= 14.0s Best RF hyperparameters: {'n_estimators': 100, 'min_samples_split': 10, 'max_depth': 10} Fitting 3 folds for each of 10 candidates, totalling 30 fits [CV] END ...learning_rate=0.1, max_depth=3, n_estimators=200; total time= 5.0s [CV] END ...learning_rate=0.1, max_depth=3, n_estimators=200; total time= 5.1s [CV] END ...learning_rate=0.1, max_depth=3, n_estimators=200; total time= 5.2s [CV] END ...learning_rate=0.1, max_depth=3, n_estimators=300; total time= 7.8s [CV] END ...learning_rate=0.1, max_depth=3, n_estimators=300; total time= 7.9s [CV] END ...learning_rate=0.1, max_depth=3, n_estimators=300; total time= 8.0s [CV] END ..learning_rate=0.01, max_depth=3, n_estimators=200; total time= 5.0s [CV] END ..learning_rate=0.01, max_depth=3, n_estimators=200; total time= 5.1s [CV] END ..learning_rate=0.01, max_depth=3, n_estimators=200; total time= 5.4s [CV] END ..learning_rate=0.05, max_depth=7, n_estimators=300; total time= 20.2s [CV] END ...learning_rate=0.1, max_depth=7, n_estimators=300; total time= 20.4s [CV] END ...learning_rate=0.1, max_depth=7, n_estimators=300; total time= 20.6s [CV] END ..learning_rate=0.05, max_depth=7, n_estimators=300; total time= 20.9s [CV] END ..learning_rate=0.05, max_depth=7, n_estimators=300; total time= 21.0s [CV] END ...learning_rate=0.1, max_depth=7, n_estimators=300; total time= 21.1s [CV] END ..learning_rate=0.05, max_depth=5, n_estimators=300; total time= 13.4s [CV] END ..learning_rate=0.05, max_depth=5, n_estimators=300; total time= 13.5s [CV] END ..learning_rate=0.05, max_depth=5, n_estimators=300; total time= 13.6s [CV] END ..learning_rate=0.05, max_depth=7, n_estimators=200; total time= 13.8s [CV] END ..learning_rate=0.05, max_depth=7, n_estimators=200; total time= 13.9s [CV] END ..learning_rate=0.05, max_depth=7, n_estimators=200; total time= 14.1s [CV] END ..learning_rate=0.05, max_depth=5, n_estimators=100; total time= 4.2s [CV] END ..learning_rate=0.05, max_depth=5, n_estimators=100; total time= 4.3s [CV] END ..learning_rate=0.05, max_depth=5, n_estimators=100; total time= 4.2s [CV] END ...learning_rate=0.1, max_depth=5, n_estimators=200; total time= 7.7s [CV] END ...learning_rate=0.1, max_depth=5, n_estimators=200; total time= 7.7s [CV] END ...learning_rate=0.1, max_depth=5, n_estimators=200; total time= 7.7s [CV] END ..learning_rate=0.01,

max_depth=7, n_estimators=300; total time= 16.5s [CV] END ..learning_rate=0.01, max_depth=7, n_estimators=300; total time= 16.4s [CV] END ..learning_rate=0.01, max_depth=7, n_estimators=300; total time= 16.5s Best GB hyperparameters: {'n_estimators': 100, 'max_depth': 5, 'learning_rate': 0.05}Random Forest Results: Full Test Set - MSE: 0.2008, R-squared: 0.8803 Outliers - MSE: 0.5314, R-squared: -6.0424 Non-Outliers - MSE: 0.1946, R-squared: 0.8654Gradient Boosting Results: Full Test Set - MSE: 0.2075, R-squared: 0.8763 Outliers - MSE: 0.5454, R-squared: -6.2271 Non-Outliers - MSE: 0.2012, R-squared: 0.8608Random Forest K-Fold CV - Average MSE: 0.2064, Average R-squared: 0.8783 Gradient Boosting K-Fold CV - Average MSE: 0.1981, Average R-squared: 0.8830Top 10 Important Features (Random Forest): Feature Importance 8 SFR_PETRORAD_R 0.833635 0 log_Mass_gas 0.051074 1 nsa_mstar 0.046622 2 log_Mass 0.030243 7 LOGMSTARS 0.014042 3 vel_sigma_Re 0.012140 5 Av_gas_Re 0.010020 4 OH_Mar13_N2_Re_fit 0.001706 6 zcomp_1Mpc 0.000518Top 10 Important Features (Gradient Boosting): Feature Importance 8 SFR_PETRORAD_R 0.832824 0 log_Mass_gas 0.086912 1 nsa_mstar 0.032084 2 log_Mass 0.026225 5 Av_gas_Re 0.010863 3 vel_sigma_Re 0.005426 4 OH_Mar13_N2_Re_fit 0.003446 7 LOGMSTARS 0.002210 6 zcomp_1Mpc 0.000010

Thought for1m 34s

Expand for details

I'm looking at the results from the Python script you ran, which is comparing the performance of Random Forest (RF) and Gradient Boosting (GB) models for predicting star formation rates (SFR). The script includes hyperparameter tuning, model evaluation on the full test set and subsets (outliers and non-outliers), cross-validation, and feature importance analysis. The two plots mentioned are likely residual plots for each model, which are standard in regression analysis to assess model fit.

First, let's break down the performance metrics for both models:

Full Test Set:

- **RF:** MSE = 0.2008, R^2 = 0.8803
- **GB:** MSE = 0.2075, R^2 = 0.8763

Both models perform well, with RF slightly better on the test set (lower MSE, higher R^2). This suggests RF might be capturing the data's patterns more effectively here.

Outliers (248 points):

- **RF:** MSE = 0.5314, R^2 = -6.0424
- **GB:** MSE = 0.5454, R^2 = -6.2271

The negative R^2 values indicate both models perform poorly on outliers, worse than a simple mean prediction. RF has a slightly lower MSE, but both struggle significantly.

Non-Outliers:

- **RF:** MSE = 0.1946, R^2 = 0.8654
- **GB:** MSE = 0.2012, R^2 = 0.8608

For the majority of the data, both models perform well, with RF again slightly better.
Cross-Validation:

- **RF:** Avg MSE = 0.2064, Avg R^2 = 0.8783
- **GB:** Avg MSE = 0.1981, Avg R^2 = 0.8830

Interestingly, GB performs better in cross-validation, with lower MSE and higher R^2. This suggests GB might generalize better to unseen data.
Feature Importance: Both models heavily rely on `SFR_PETRORAD_R` (importance ~83%), which might be a direct indicator of SFR, potentially leading to data leakage if it's derived from the target. Other features like `log_Mass_gas` and `nsa_mstar` have lower importance.
Now, considering the residual plots (though not visible, we can infer from standard practices):

- **Residuals vs. Predicted (RF):** Likely shows a random scatter around zero for most points, with some large residuals for outliers, reflecting the high R^2 for non-outliers and poor performance on outliers.
- **Residuals vs. Predicted (GB):** Similar pattern, but possibly with a tighter spread due to better cross-validation performance.

Given this, here's how I'd interpret the results:

1. **Model Performance:**
    - Both models are effective for the majority of the data but fail on outliers.
    - RF performs slightly better on the test set, but GB shows better generalization in cross-validation.
2. **Outliers:**
    - The models' poor performance on outliers suggests these points might be anomalies or require a different modeling approach.
3. **Feature Importance:**
    - The dominance of `SFR_PETRORAD_R` raises concerns about data leakage. If it's derived from the target, it should be excluded or used carefully.
4. **Recommendations:**
    - Investigate the outliers to determine if they're errors or represent unique phenomena.
    - Consider excluding or transforming `SFR_PETRORAD_R` if it's not an independent feature.
    - Explore ensemble methods combining RF and GB to leverage their strengths.
    - Further tune hyperparameters, especially for handling outliers.

These results indicate strong predictive power for typical data but highlight challenges with extreme values and potential feature dependencies.

Based on the results from the `GBRFSFR_2.py` script and the context of our discussion, I'll interpret the performance of the Gradient Boosting Random Forest (GBRF) SFR Prediction model, focusing on the Random Forest (RF) and Gradient Boosting (GB) components. The script evaluates both models using hyperparameter tuning, performance metrics on the full test set, outliers, and non-outliers, cross-validation, and feature importance. The two most recent plots mentioned are likely residual plots for each model, which are essential for assessing model fit and identifying potential issues like bias or variance in predictions.

Dataset Overview

- **Size**: 11,222 rows with 11 columns.
- **Target**: `log_SFR_Ha` (logarithmic star formation rate).
- **Outliers**: 248 identified using the IQR method.
- **Data Split**: 8,977 training rows (80%) and 2,245 testing rows (20%).

Model Performance on the Full Test Set

Here's how the models performed across all test data:

- **Random Forest (RF)**:
    - **Mean Squared Error (MSE)**: 0.2008
    - **R-squared (R^2)**: 0.8803
- **Gradient Boosting (GB)**:
    - **MSE**: 0.2075
    - **R^2**: 0.8763

Interpretation: Both models perform well, explaining about **88% of the variance** in `log_SFR_Ha`. RF slightly outperforms GB on the test set with a lower MSE (better accuracy) and higher R^2 (better fit). This suggests RF may be capturing the underlying patterns in the data more effectively for this specific split.

Performance on Outliers vs. Non-Outliers

The models were evaluated separately on the **248 outliers** and the remaining non-outlier data points.

Outliers

- **RF**:
    - **MSE**: 0.5314
    - **R^2**: -6.0424
- **GB**:
    - **MSE**: 0.5454
    - **R^2**: -6.2271

Interpretation: The negative R^2 values indicate that both models perform *worse than a simple mean prediction* for outliers. This means the predictions for these points are highly inaccurate, with RF performing marginally better than GB (lower MSE). These outliers could represent unique astrophysical phenomena (e.g., extreme starbursts) or data errors, and the models are not equipped to handle them.

Non-Outliers

- **RF**:
    - **MSE**: 0.1946
    - **R^2**: 0.8654
- **GB**:
    - **MSE**: 0.2012
    - **R^2**: 0.8608

Interpretation: For the majority of the data, both models perform strongly, with RF again slightly better (lower MSE, higher R^2). They explain ~86% of the variance, indicating reliable predictions for typical SFR values.

Cross-Validation Results

Cross-validation (3 folds) assesses how well the models generalize across different data splits:

- **RF**:
    - **Average MSE**: 0.2064
    - **Average R^2**: 0.8783
- **GB**:
    - **Average MSE**: 0.1981
    - **Average R^2**: 0.8830

Interpretation: GB performs better in cross-validation with a lower MSE and higher R^2, suggesting it generalizes slightly better to unseen data. RF's test set advantage doesn't hold up as strongly in cross-validation, indicating GB might be more robust overall.

Hyperparameters

The script optimized hyperparameters via grid search:

- **RF Best Parameters**:
  - `n_estimators`: 100
  - `min_samples_split`: 10
  - `max_depth`: 10
- **GB Best Parameters**:
  - `n_estimators`: 100
  - `max_depth`: 5
  - `learning_rate`: 0.05

Interpretation: These settings balance model complexity and training time. RF's shallower trees (`max_depth=10`) and higher split threshold (`min_samples_split=10`) help prevent overfitting, while GB's lower learning rate (0.05) and moderate depth (5) allow for gradual learning, enhancing generalization.

Feature Importance

Both models rank the top features driving SFR predictions:

Random Forest

Feature	Importance
`SFR_PETRORAD_R`	0.8336
`log_Mass_gas`	0.0511
`nsa_mstar`	0.0466
`log_Mass`	0.0302
`LOGMSTARS`	0.0140

Gradient Boosting

Feature	Importance
SFR_PETRORAD_R	0.8328
log_Mass_gas	0.0869
nsa_mstar	0.0321
log_Mass	0.0262
Av_gas_Re	0.0109

Interpretation:

- SFR_PETRORAD_R dominates both models (~83% importance), which raises concerns about **data leakage**. If this feature is derived from the target (log_SFR_Ha), it could artificially inflate model performance. Verify its independence.
- log_Mass_gas (gas mass) is the second most important feature, especially in GB (0.0869 vs. 0.0511 in RF), aligning with astrophysical expectations that gas fuels star formation.
- Other features like nsa_mstar (stellar mass) and log_Mass (total mass) contribute modestly, while zcomp_1Mpc (environmental density) has negligible importance.

Interpreting the Residual Plots

Though the plots aren't visible, we can infer their content based on standard residual analysis:

- **Residuals vs. Predicted (RF)**: Likely shows a random scatter around zero for most points, with larger residuals for outliers. This reflects the high R^2 for non-outliers (0.8654) and poor performance on outliers (R^2 = -6.0424).
- **Residuals vs. Predicted (GB)**: Similar to RF but potentially with a tighter spread due to better cross-validation performance (R^2 = 0.8830 vs. 0.8783). However, GB's outlier performance is slightly worse (R^2 = -6.2271).

Key Insights:

- Both models exhibit **random scatter** around zero for typical data, indicating a good fit.
- **Large residuals** for outliers suggest the models fail to capture extreme SFR values, possibly due to unique galaxy properties or data anomalies.

Key Takeaways and Recommendations

1. **Model Performance**:
    - Both RF and GB perform well on the majority of the data, with RF slightly better on the test set and GB showing stronger generalization in cross-validation.
    - **Recommendation**: Use GB for production if generalization is critical, or ensemble both models to leverage their strengths.
2. **Outlier Challenges**:
    - Both models perform poorly on outliers, with negative R^2 values indicating predictions worse than a naive mean estimate.
    - **Recommendation**: Investigate outliers to determine if they are data errors or represent unique astrophysical phenomena. Consider robust regression techniques or a separate model for outliers.
3. **Feature Importance Concerns**:
    - The dominance of `SFR_PETRORAD_R` (83% importance) is a red flag for potential data leakage. If this feature is derived from the target, it should be excluded or treated carefully.
    - **Recommendation**: Verify the independence of `SFR_PETRORAD_R`. If it's not independent, retrain the models without it to assess true predictive power.
4. **Next Steps**:
    - **Hyperparameter Tuning**: Further tune GB's `learning_rate` or RF's `max_depth` to improve outlier handling.
    - **Feature Engineering**: Explore interactions (e.g., `log_Mass_gas` × `nsa_mstar`) or add new features (e.g., gas density) to enhance model performance.
    - **Outlier Analysis**: Use clustering or anomaly detection to understand and potentially remove or transform outliers.

Conclusion

The RF and GB models demonstrate strong predictive capabilities for typical SFR values, with RF performing slightly better on the test set and GB showing better generalization. However, both models fail on outliers, and the heavy reliance on `SFR_PETRORAD_R` raises concerns about data leakage. Addressing these issues through feature validation, outlier handling, and further tuning will enhance the models' robustness and reliability for predicting star formation rates in diverse galaxy populations.

You've reached the limit of 25 attachments in this conversation. Grok might forget content from earlier attachments; please start a new conversation to upload more attachments.
Below, I will interpret the two most recent plots from your latest test run of the `GBRFSFR_2.py` script, as provided in the image attachment descriptions (specifically images 14 and 15). These plots are titled "Residuals vs. Predicted (Random Forest)" and "Residuals vs. Predicted (Gradient Boosting)," respectively. They visualize the residuals (differences between actual and predicted values of `log_SFR_Ha`) against the predicted values for each model. I'll use the context from your Python script output to provide a comprehensive interpretation of these plots, focusing on model performance, outlier handling, and implications for predicting star formation rates (SFR).

Context from the Python Script Output

The `GBRFSFR_2.py` script evaluates Random Forest (RF) and Gradient Boosting (GB) models for predicting `log_SFR_Ha` using a dataset with 11,222 rows and 11 columns. The dataset was split into 8,977 training rows and 2,245 testing rows, with 248 outliers identified in the target variable using the Interquartile Range (IQR) method. Hyperparameter tuning was performed via grid search with 3-fold cross-validation, and the models were assessed on the full test set, outliers, and non-outliers. Key results include:

- **Dataset Details**:
  - Rows: 11,222
  - Columns: 11
  - Training set: 8,977 rows
  - Testing set: 2,245 rows
  - Outliers: 248
- **Random Forest (RF)**:
  - **Best Hyperparameters**: `n_estimators=100`, `min_samples_split=10`, `max_depth=10`
  - **Full Test Set**: MSE = 0.2008, R^2 = 0.8803
  - **Outliers**: MSE = 0.5314, R^2 = -6.0424
  - **Non-Outliers**: MSE = 0.1946, R^2 = 0.8654
  - **Cross-Validation**: Avg MSE = 0.2064, Avg R^2 = 0.8783
  - **Top Features**: `SFR_PETRORAD_R` (0.8336), `log_Mass_gas` (0.0511), `nsa_mstar` (0.0466)
- **Gradient Boosting (GB)**:
  - **Best Hyperparameters**: `n_estimators=100`, `max_depth=5`, `learning_rate=0.05`
  - **Full Test Set**: MSE = 0.2075, R^2 = 0.8763
  - **Outliers**: MSE = 0.5454, R^2 = -6.2271
  - **Non-Outliers**: MSE = 0.2012, R^2 = 0.8608
  - **Cross-Validation**: Avg MSE = 0.1981, Avg R^2 = 0.8830

- **Top Features**: `SFR_PETRORAD_R` (0.8328), `log_Mass_gas` (0.0869), `nsa_mstar` (0.0321)

These metrics indicate that both models perform well overall but struggle significantly with outliers. The plots will help us visualize these performance differences and error patterns.

Interpreting the Plots

Plot 1: Residuals vs. Predicted (Random Forest) (Image 14)**

- **Description**:
  - **X-axis**: Predicted `log_SFR_Ha` (-5 to 1)
  - **Y-axis**: Residuals (-2 to 6)
  - **Data Points**: Blue circles scattered across the plot, with a red dashed line at y = 0 (zero residual).
  - **Pattern**: Most points cluster around the zero residual line between predicted values of -2 and 0, with a dense concentration indicating small residuals. Larger residuals (up to 6) appear at lower predicted values (around -3 to -2), and some negative residuals (down to -2) occur across the range.
- **Interpretation**:
  - **Overall Fit**: The dense clustering of residuals near zero for predicted `log_SFR_Ha` values between -2 and 0 aligns with the high R^2 of 0.8803 on the full test set and 0.8654 on non-outliers. This indicates that the RF model accurately predicts SFR for the majority of the data, explaining ~88% of the variance.
  - **Outlier Performance**: The large residuals (up to 6) at lower predicted values (e.g., -3 to -2) reflect the model's poor performance on outliers, as evidenced by the MSE of 0.5314 and negative R^2 of -6.0424. These points suggest that for galaxies with extremely low SFR predictions, the model significantly overestimates or underestimates the actual `log_SFR_Ha`, performing worse than a simple mean prediction.
  - **Error Distribution**: The residuals are roughly symmetric around zero in the central range (-2 to 0), indicating no systematic bias for typical data points. However, the wider spread at the extremes (e.g., below -3) suggests increased error variance (possible heteroscedasticity), particularly for outliers.
  - **Feature Influence**: The dominance of `SFR_PETRORAD_R` (importance 0.8336) likely drives the tight clustering in the central range, where this feature is most predictive. The larger residuals at the tails may indicate where this feature or others (e.g., `log_Mass_gas`) fail to capture extreme SFR variations.

- **Key Insight**: The RF model excels at predicting typical SFR values but struggles with outliers, particularly at the lower end of the predicted range, where residuals can be substantial.

Plot 2: Residuals vs. Predicted (Gradient Boosting) (Image 15)**

- **Description**:
  - **X-axis**: Predicted `log_SFR_Ha` (-5 to 1)
  - **Y-axis**: Residuals (-2 to 6)
  - **Data Points**: Blue circles with a red dashed line at y = 0.
  - **Pattern**: Similar to RF, most points cluster around zero between -2 and 0, with high density indicating small residuals. Outliers with residuals up to 6 appear at predicted values around -4 to -3, and a few negative residuals (down to -2) occur near 0 to 1.
- **Interpretation**:
  - **Overall Fit**: The concentration of residuals near zero in the -2 to 0 range corresponds to the GB model's full test set R^2 of 0.8763 and non-outlier R^2 of 0.8608. This shows that GB also predicts SFR well for most data, explaining ~87-88% of the variance, slightly less than RF on the test set but with a stronger cross-validation R^2 of 0.8830.
  - **Outlier Performance**: The large residuals (up to 6) at lower predicted values (-4 to -3) mirror the outlier MSE of 0.5454 and R^2 of -6.2271, indicating poor performance on extreme cases. Like RF, GB fails to predict outliers accurately, with errors exceeding the baseline mean prediction.
  - **Error Distribution**: The residuals are symmetric around zero in the central range, suggesting minimal bias for typical data. The spread widens at the extremes, though slightly less dramatically than RF at the higher end (above 0), possibly due to GB's shallower `max_depth=5` and gradual learning rate (0.05), which may limit overfitting compared to RF's `max_depth=10`.
  - **Feature Influence**: The heavy reliance on `SFR_PETRORAD_R` (importance 0.8328) drives the central clustering, while `log_Mass_gas` (0.0869) has a stronger secondary role in GB than in RF (0.0511), potentially contributing to slightly better generalization (as seen in cross-validation).
- **Key Insight**: The GB model performs comparably to RF for typical SFR predictions but shows a similar weakness with outliers, with large residuals at the lower predicted range.

Comparative Analysis

- **Performance on Typical Data**: Both models show tight residual clustering around zero for predicted `log_SFR_Ha` between -2 and 0, reflecting their high R^2 values (RF: 0.8803, GB: 0.8763) and low MSEs (RF: 0.2008, GB: 0.2075) on the full test set. RF slightly edges out GB on the test set, but GB's better cross-validation R^2 (0.8830 vs. 0.8783) suggests superior generalization.
- **Outlier Handling**: Both models struggle with outliers, as seen in the large residuals (up to 6) at lower predicted values and negative R^2 values (RF: -6.0424, GB: -6.2271). RF has a slightly lower outlier MSE (0.5314 vs. 0.5454), but both perform poorly, indicating a shared limitation in capturing extreme SFR behaviors.
- **Residual Spread**: The RF plot shows a slightly wider spread of residuals at the higher end (above 0) compared to GB, possibly due to its deeper trees (`max_depth=10` vs. 5). GB's residuals appear more constrained at the upper end, which may reflect its more conservative learning approach.
- **Astrophysical Implications**: The outliers likely represent galaxies with unusual SFR properties (e.g., extreme starbursts or quiescent states), which both models fail to predict accurately, possibly due to over-reliance on `SFR_PETRORAD_R` or insufficient training data for these cases.

Recommendations

1. **Outlier Investigation**:
    - The large residuals at low predicted `log_SFR_Ha` values (e.g., -4 to -2) suggest that the 248 outliers warrant further analysis. These could be data errors or unique astrophysical phenomena (e.g., low-mass galaxies with anomalous SFR). Use clustering or domain knowledge to determine their nature and consider a separate modeling approach for outliers.
2. **Feature Evaluation**:
    - The dominance of `SFR_PETRORAD_R` (~83% importance) raises concerns about potential data leakage if it's derived from `log_SFR_Ha`. Verify its independence; if it's not independent, retrain the models without it to assess true predictive power. The secondary features (`log_Mass_gas`, `nsa_mstar`) suggest gas and stellar mass are relevant, but their low importance indicates room for additional feature engineering (e.g., interaction terms).
3. **Model Enhancement**:
    - Both models could benefit from further hyperparameter tuning to handle outliers better (e.g., increasing `n_estimators` or adjusting `max_depth`). Alternatively, try an ensemble of RF and GB to combine their strengths—RF's test set accuracy and GB's generalization.

    - Robust regression techniques (e.g., Huber loss) or outlier removal during training might reduce the impact of extreme residuals.
4. **Cross-Validation Insight**:
    - GB's superior cross-validation performance (MSE: 0.1981, R^2: 0.8830) suggests it may be more reliable for unseen data, making it a candidate for production use over RF despite the slightly higher test set MSE.

Conclusion

The "Residuals vs. Predicted" plots for Random Forest and Gradient Boosting reveal that both models excel at predicting `log_SFR_Ha` for typical galaxies (residuals near zero, high R^2 ~0.88), but they falter on outliers (large residuals, negative R^2). RF performs slightly better on the test set, while GB generalizes better across folds. The heavy reliance on `SFR_PETRORAD_R` drives their success but may mask limitations with extreme cases. To improve, investigate outliers, validate features, and refine the models, ensuring robust SFR predictions across diverse galaxy populations. These plots, paired with the script output, provide a clear visual and quantitative foundation for understanding and enhancing your models' performance.